Model Checking Lecture Notes

Sebastian Junges

Saturday 23rd May, 2026

1 Markov Decision Processes

1.1 What are Markov decision processes?discuss!

Markov decision processes are a state-based model that combine nondeterministic action choices with probabilistic action outcomes.

1.1.1 Formal definitionsdiscuss!

Definition 1.1 (Markov Decision Process)**.** *An MDP M is a tuple*

$$\langle S, A, \delta \rangle$$

where

- *S is a nonempty set of states,*
- *A is a nonempty set of actions,*
- *$\delta \colon S \times A \nrightarrow \mathsf{Distr}(S)$ is a partial transition function.*

Additionally, we often assume the existence of a unique *initial state* s_{init}. Later, we extend MDPs with *atomic propositions*, *target states*, and *reward functions*.

Definition 1.2 (Enabled actions)**.** *For any state s, the set of enabled actions is*

$$A_{\mathsf{en}}(s) = \{a \mid \delta(s,a) \neq \bot\}. \tag{1}$$

We use the notion of a *choice* to denote a state–action pair $\langle s, a \rangle$ where $a \in A_{\mathsf{en}}(s)$.

Markov chains are MDPs with exactly one choice per state. We can then omit actions and write a Markov chain as a tuple $\langle S, \delta \rangle$, potentially extended by initial states.

For any choice $\langle s, a \rangle$, we may write $\delta(s, a, s')$ to denote $\delta(s, a)(s')$.

Assumptions
Unless stated otherwise, we assume

- S and A are finite,
- there are no *deadlocks*: $A_{\mathsf{en}}(s) \neq \emptyset$, and
- all probabilities $\delta(s,a)(s')$ are rational.

Definition 1.3 (Path)**.** *A path is a sequence $s_0 a_0 s_1 \cdots \in (S \times A)^* \times S$ such that for all i:*

- $\langle s_i, a_i \rangle$ *is a choice, and*
- $\delta(s_i, a_i)(s_{i+1}) > 0$.

The set of all paths is denoted Paths^M. *We drop* M *whenever it is clear from the context. We write* $\mathsf{Paths}^M(s)$ *for the set of paths with* $s_0 = s$, *and* $\mathsf{Paths}(s, T)$ *for paths that start in* s *and end in* T.

For a finite path $\xi = s_0 a_0 s_1 \dots s_n$ we write $\mathsf{last}(\xi)$ for the final state s_n.
For an infinite path $\xi = s_0 a_0 s_1 a_1 s_2 \dots$, we define

$$\mathsf{inf}(\xi) = \{\langle s, a \rangle \mid |\{i \mid \langle s_i, a_i \rangle = \langle s, a \rangle\}| = \infty\} \tag{2}$$

as the choices that are made infinitely often along a path.

Graph structurediscuss! A *sink state* is a state s with $\delta(s, a)(s) = 1$ for all $a \in A_{\mathsf{en}}(s)$.

An MDP is *acyclic* if along every infinite path, the only states visited infinitely often are sink states.

Policiesdiscuss! Policies resolve nondeterminism.

> **Note**
> In the literature, different words are used. In the classical model checking literature, the word *scheduler* is preferred, in game theory, the word *strategy* is preferred, and in control theory, policies are often called *controllers*.

Definition 1.4 (Policy)**.** *For an MDP* M, *a policy is a function*

$$\pi \colon \mathsf{Paths}^M \to \mathsf{Distr}(A).$$

- *A policy* π *is memoryless if* $\pi(\xi) = \pi(\xi')$ *for all paths with the same last state.*
- *A policy is deterministic if every distribution* $\pi(\xi)$ *is Dirac.*

We denote the set of all policies by Π. We use Π_{md} for the memoryless deterministic policies and Π_{mr} for the memoryless (randomising) policies. Memoryless deterministic policies can be written as $\pi \colon S \to A$, memoryless randomising as $\pi \colon S \to \mathsf{Distr}(A)$.

We furthermore define the set of paths under a policy Paths^π as the set of paths that are consistent with a policy π.

Induced Markov Chainsdiscuss! Once a policy resolves nondeterminism in an MDP, the behaviour becomes purely probabilistic and can thus be captured by a Markov chain.

Definition 1.5 (Induced Markov chain (general policies))**.** *Given MDP M and policy π, the induced Markov chain $M[\pi]$ is $\langle \mathsf{Paths}^M, \delta^\pi \rangle$ with*

$$\delta^\pi(\xi)(\xi \cdot as) = \pi(\xi)(a)\, \delta(\mathsf{last}(\xi), a)(s).$$

Remark 1.1 (Infinite Markov chain)**.** *The above Markov chain is infinite, but reachability probabilities remain well defined.*

It is useful to consider a special case for memoryless (possibly randomised) policies.

Definition 1.6 (Induced Markov chain (memoryless policies))**.** *For a memoryless policy π, the induced Markov chain is $M[\pi] = \langle S, \delta^\pi \rangle$ where*

$$\delta^\pi(s)(s') = \sum_a \pi(s)(a)\delta(s,a)(s').$$

Remark 1.2 (Two definitions?)**.** *These two constructions yield equivalent Markov chains (formally: almost bisimilar).*

Definition 1.7. *Given an MDP M and a set of policies Π, we define*

$$M \mid \Pi = \{M[\pi] \mid \pi \in \Pi\}. \tag{3}$$

We define

$$M_{\mathsf{MD}} = M \mid \Pi_{md} \text{ and } M_{\mathsf{MR}} = M \mid \Pi_{mr} \tag{4}$$

1.2 Reachability probabilitiesdiscuss!

For a policy π, the *reachability probability* $\mathsf{Pr}^\pi_M(s \models \Diamond T)$ denotes the probability of eventually reaching a target state T from a state s. The probability $\mathsf{Pr}^\pi_M(s \models \Diamond T)$ is defined via the induced MC $M[\pi]$. We are mostly interested in this probability from the initial state s_{init}, in which case we simplify notation to $\mathsf{Pr}^\pi_M(\Diamond T)$.

Problem: Standard verification problem for reachability probabilities
Given MDP M, a *threshold* $\lambda \in \mathbb{Q}$ and $\bowtie \in \{\leq, \geq\}$, decide whether

$$\mathsf{Pr}^\pi_M(\Diamond T) \bowtie \lambda \tag{5}$$

holds for **all** policies.

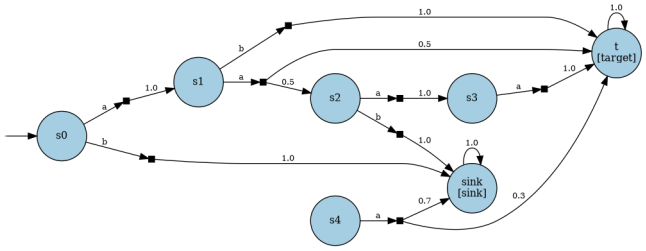
Problem: Standard planning problem for reachability probabilities
Given MDP M, $\lambda \in \mathbb{Q}$ and $\bowtie$, compute **some** policy π such that

$$\mathsf{Pr}_M^\pi(\Diamond T) \bowtie \lambda. \tag{6}$$

1.2.1 Qualitative verificationdiscuss!

Qualitative verification refers to the setting where the threshold λ is either zero or one. As we will see, qualitative verification can be solved without reference to the precise probabilities in the MDP. Assume fixed M and target set T.

Example 1.1 (Qualitative verification - running example)**.** *Throughout this section, we use the following MDP as a running example.*



Possible reachability (max)discuss! One of the simplest questions we can ask is to find the set of states that can reach the targets with positive probability.

More formally, we denote this set of states

$$S_{>0}^{max} = \{s \mid \exists \pi \text{ s.t. } \mathsf{Pr}^\pi(s \models \Diamond T) > 0\} \tag{7}$$

, i.e., the set of states where the *maximum* probability over all policies is positive. It holds that

$S_{>0}^{max} = S \setminus S_{=0}^{max}$ using

$$S_{=0}^{max} = \{s \mid \exists \pi \text{ s.t. } \mathsf{Pr}^\pi(s \models \Diamond T) = 0\}. \tag{8}$$

The notation contrasts with $S_{=0}^{min} = \{\forall \pi \text{ s.t. } \mathsf{Pr}^\pi(s \models \Diamond T) = 0\}$ where the *minimum* probability over all policies is zero and which we discuss below.

Theorem 1.1. *For every state s,*

$$s \in S_{>0}^{max} \quad \textit{iff} \quad \exists \xi \in \mathsf{Paths}(s, T). \tag{9}$$

The main consequence is that computing $S_{>0}^{max}$ reduces to graph reachability. We omit a technical proof of the theorem and focus on the intuition:

- If there is a path in the MDP, then there is a policy that takes exactly the choices in that path. Pick such a policy. In the induced Markov chain, the same path exists, and thus, the probability is positive.
- If there is no such path in the MDP, then a policy cannot make choices that lead to a path in the induced Markov chain.

This can be computed as a least fixpoint. Specifically, we define

$$\Psi_{\max>0} \colon 2^S \to 2^S \tag{10}$$

such that

$$\Psi_{\max>0}(X) = \{s \in S \mid \exists a \in A_{\mathsf{en}}(s) \,.\, \exists s' \in X.\, \delta(s,a)(s') > 0\} \cup T \tag{11}$$

The operator is monotonic and the lattice is finite.

Theorem 1.2. $\mathsf{lfp}.\Psi_{\max>0} = S^{max}_{>0}$.

Example 1.2. *Consider Example 1.1. Starting from $X_0 = T$, the operator $\Psi_{\max>0}$ adds all states with any action reaching the current set in one step.*

	iter 0	*iter 1*	*iter 2*	*iter 3*
state				
s0	*False*	*False*	*True*	*True*
s1	*False*	*True*	*True*	*True*
s2	*False*	*False*	*True*	*True*
s3	*False*	*True*	*True*	*True*
s4	*False*	*True*	*True*	*True*
sink	*False*	*False*	*False*	*False*
t	*True*	*True*	*True*	*True*

Possible reachability (min)discuss! We now study computing

$$S^{min}_{>0} = \{\forall \pi \text{ s.t. } \mathsf{Pr}^{\pi}(s \models \Diamond T) > 0\}, \tag{12}$$

that is, the complement of $S^{min}_{=0}$. Concretely, we will use that each policy reaches the target with positive probability iff it has a path to the target of length n. This can be computed as a least fixpoint. Specifically, we define

$$\Psi_{\min>0} \colon 2^S \to 2^S \tag{13}$$

such that

$$\Psi_{\min>0}(X) = \{s \in S \mid \text{for all } a \in A_{\mathsf{en}}(s)\, \exists s' \in X.\delta(s,a)(s') > 0\} \cup T \tag{14}$$

The operator is monotonic and the lattice is finite.

Theorem 1.3. $\mathsf{lfp}.\Psi_{\min>0} = S^{min}_{>0}$.

Example 1.3. *Consider Example 1.1. Starting from $X_0 = T$, the operator $\Psi_{\min>0}$ adds states where every action has at least one successor already in the current set.*

	iter 0	*iter 1*	*iter 2*
state			
s0	*False*	*False*	*False*
s1	*False*	*True*	*True*
s2	*False*	*False*	*False*
s3	*False*	*True*	*True*
s4	*False*	*True*	*True*
sink	*False*	*False*	*False*
t	*True*	*True*	*True*

Almost-sure reachability (max)discuss! We are also interested in computing the set of states from which it is possible to ensure that we almost-surely reach the target states.

More formally, we denote this set of states

$$S^{max}_{=1} = \{s \mid \exists \pi \text{ s.t. } \mathsf{Pr}^{\pi}(s \models \Diamond T) = 1\}. \tag{15}$$

We use a recursive equation to characterise this set. If $s \in T$, then clearly $s \in S^{max}_{=1}$. Otherwise, for $s \notin T$:

$$s \in S^{max}_{=1} \quad \text{iff} \quad \exists a \in A_{\mathsf{en}}(s).\forall s' \in \mathsf{supp}(\delta(s,a)).s' \in S^{max}_{=1}. \tag{16}$$

To compute the set $S^{max}_{=1}$, we want to provide an iterative procedure, which operates on sets of states. Specifically, we define the operator

$$\Psi_{\max=1} \colon 2^S \to 2^S \tag{17}$$

such that

$$\Psi_{\max=1}(X) = \{s \in S^{max}_{>0} \mid \exists a \in A_{\mathsf{en}}(s).\forall s' \in \mathsf{supp}(\delta(s,a)).s' \in X\} \cup T \tag{18}$$

The operator is monotonic and the lattice is finite.

Theorem 1.4. $\mathsf{gfp}.\Psi_{\max=1} = S^{max}_{=1}$.

Restricting the operator's co-domain to $S^{max}_{>0}$ is essential. Without it, S itself is always a fixed point: any non-target state with a self-loop has an action whose only successor is itself, so it is never removed. By restricting to $S^{max}_{>0}$, states from which T is unreachable are excluded before the GFP begins.

An equivalent formulation (see Baier and Katoen [2008], Algorithm 45) operates at *action* granularity: actions whose support intersects $S \setminus X$ are removed

from the MDP, and a state is only removed once all its actions are gone. The two formulations agree because the GFP sequence is decreasing — once an action's successor leaves the current set, the successor never returns — so the set of invalidated actions grows monotonically, exactly as in the action-removal algorithm. Our set-based formulation is more concise; the action-removal view is closer to an efficient implementation.

Example 1.4. *Consider Example 1.1. The GFP starts from $S_{>0}^{max}$ (states that can possibly reach T) and removes states that lack any action with all successors inside the current set.*

	iter 0	*iter 1*	*iter 2*
state			
s0	*True*	*True*	*True*
s1	*True*	*True*	*True*
s2	*True*	*True*	*True*
s3	*True*	*True*	*True*
s4	*True*	*False*	*False*
sink	*False*	*False*	*False*
t	*True*	*True*	*True*

Almost-sure reachability (min)discuss! Finally, we can also define the set of states

$$S_{=1}^{min} = \{s \mid \forall \pi \text{ s.t. } \mathsf{Pr}^{\pi}(s \models \Diamond T) = 1\}. \tag{19}$$

> **Attention**
> Not discussed in the lecture.

1.2.2 Quantitative reachabilitydiscuss!

We are interested in computing the

- maximal reachability probability

$$\sup_{\pi \in \Pi} \mathsf{Pr}^{\pi}(s \models \Diamond T), \tag{20}$$

and the

- minimal reachability probability

$$\inf_{\pi \in \Pi} \mathsf{Pr}^{\pi}(s \models \Diamond T). \tag{21}$$

Note
The use of minimal and maximal rather than infimum and supremum in the naming is adequate due to Theorem 1.6 and Theorem 1.8 respectively.

While both cases are mostly symmetrical, the minimal reachability probability is a bit easier to compute.

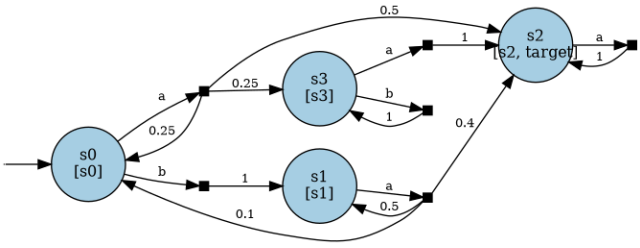
Minimal reachability probabilitydiscuss!

Theorem 1.5 (Bellman equations (MinReachProb))**.** ***Baier and Katoen [2008]***
Given an MDP with states S. Consider variables x_s for each $s \in S$. The minimal reachability probability is the unique solution to the following equation system

$$x_s = \begin{cases} 1 & s \in T \\ 0 & s \in S^{min}_{=0} \\ \min_{a \in A_{en}(s)} \sum_{s'} \delta(s,a,s') x_{s'} & otherwise \end{cases} \tag{22}$$

These equations are called the Bellman equations for minimal reachability probabilities.

Figure 1: Visualisation of an MDP to illustrate the Theorem 1.5.



Example 1.5. *Consider the MDP from Figure 1. The Bellman equations for minimal reachability probability are:*

<IPython.core.display.Math object>

The equations have a unique solution.

Bellman equations vs Bellman operators
We will later also talk about Bellman operators. At that point, we will clarify the difference.

Different Bellman equations
We will see various different Bellman equations. It is important to clarify which Bellman equations one is talking about.

We observe the following:

- If there is only one choice per state, the equation system trivially reduces to the linear equation system for Markov chains.
- Using the preprocessing with $S^{min}_{=1}$, we could extend the set of states for which we can ensure $x_s = 1$.
- We must ensure that states that cannot reach the target have a zero probability.

Theorem 1.6 (Memoryless policies suffice (MinReachProb))**.** *For any MDP and target set T:*

1. *For any state s it holds that*

$$\inf_{\pi \in \Pi} \mathsf{Pr}^{\pi}(s \models \Diamond T) = \min_{\pi \in \Pi_{md}} \mathsf{Pr}^{\pi}(s \models \Diamond T). \tag{23}$$

2. *There exists a policy π^* such that for all $s \in S$:*

$$\mathsf{Pr}^{\pi^*}(s \models \Diamond T) = \min_{\pi \in \Pi_{md}} \mathsf{Pr}^{\pi}(s \models \Diamond T). \tag{24}$$

For any solution to the Bellman equations, it is simple to extract **a** witnessing memoryless deterministic policy π^*—one simply takes

$$\pi^*(s) =_a \sum_{s'} \delta(s,a)(s') x_{s'}. \tag{25}$$

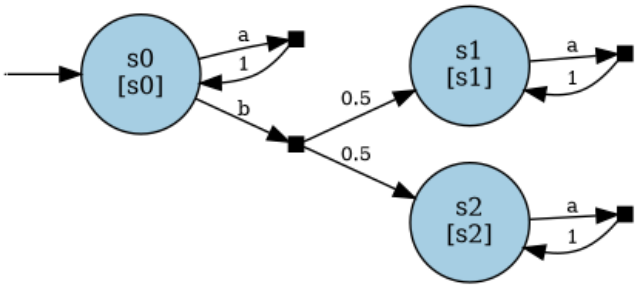
We highlight that there is a unique solution to the Bellman equations, but not a unique minimising policy. Furthermore, the theorem justifies talking about minimising policies for a target set, but independently of a specific state.

Maximal reachability probabilitydiscuss! While most aspects of computing minimal and maximal reachability probabilities are analogous, the theorems about the Bellman equations differ for both cases.

Theorem 1.7 (Bellman equations (MaxReachProb))**.** *Given an MDP with states S. Consider variables x_s for each $s \in S$. The maximal reachability probability equals the minimal solution*

$$x_s = \begin{cases} 1 & s \in T \\ 0 & s \in S^{max}_{=0} \\ \max_{a \in A_{en}(s)} \sum_{s'} \delta(s,a,s') x_{s'} & \text{otherwise} \end{cases} \tag{26}$$

Figure 2: Visualisation of an MDP with no unique solution for Theorem 1.7.



Example 1.6. *Consider the MDP in Figure 2. The Theorem 1.7 are:*

<IPython.core.display.Math object>

In particular, any assignment to $x_0 \geq 0.5$ is part of a valid solution. However, Theorem 1.7 clarifies that only $x = 0.5$ is a valid solution.

In general, the problem we observe here is that there are (sets of) states where a policy can choose to stay indefinitely. If these states do not include target states, then staying there forever means that the states have reachability probability 0, yet the Bellman equations do not enforce that these states must be assigned to zero. In Maximal end componentsdiscuss!, these sets of states are called end-components and we show that (roughly) in MDPs without such end-components, the Theorem 1.10.

Theorem 1.8 (Memoryless policies suffice (MaxReachProb))**.** *For any MDP and target set T:*

1. *For any state s it holds that*

$$\sup_{\pi \in \Pi} \mathsf{Pr}^{\pi}(\Diamond T) = \max_{\pi \in \Pi_{md}} \mathsf{Pr}^{\pi}(\Diamond T) \tag{27}$$

2. *There exists a policy π^* such that for all $s \in S$:*

$$\mathsf{Pr}^{\pi^*}(s \models \Diamond T) = \min_{\pi \in \Pi_{md}} \mathsf{Pr}^{\pi}(s \models \Diamond T). \tag{28}$$

As there is no unique solution to the Bellman equations when maximising, extracting an optimal policy is harder. In particular, while any optimal policy π must satisfy

$$\pi(s) =_a \sum_{s'} \delta(s,a)(s') x_{s'}, \tag{29}$$

this condition is not sufficient.

Example 1.7. *Consider Example 1.6. Both actions in s_0 will yield value 0.5.*

Instead, we must find a policy that makes progress towards the targets.

> **Attention**
> Content missing.

The notion of valuediscuss! To simplify the exposition, it is customary to merge discussions about minimal and maximal reachability probabilities to a fixed set of target states T. If we know from context that we want to compute the (minimal or maximal) reachability probability from state s for a set of target states T, we can call

- the probability induced by a policy *the value of the policy* (from state s),
- the minimal (resp. maximal) probability from a state s *the value of* s,
- the value of the initial state is the *value of the MDP*.
- We use $V^{\pi}_{M}(s)$ to define the value induced by a policy and $V^{*}_{M}(s)$ to define the value of a state. We omit M whenever possible.

> **Implicit notation**
> The use of values of MDPs etc is always contextual, and this is never clear from the notation. We sometimes write V^{min} or V^{max} instead of V^{*} to be more explicit.

> **Value beyond reachability probabilities**
> Later, we will also use value for other properties, including (discounted or total) expected rewards, and the probabilities for temporal properties.

1.2.3 Maximal end componentsdiscuss!

End-components are sub-MDPs induced by a set of choices (state-action pairs) where a policy can visit every state and stay forever.

Definition 1.8. *An end-component for an MDP $M = \langle S, A, \delta \rangle$ is a set $X \subseteq S \times A$ such that:*

- *X induces the submdp $M[X] = \langle S', A', \delta' \rangle$ with*
  - *$S' = \{s \mid \langle s, a \rangle \in X\}$,*
  - *$A' = \{a \mid \langle s, a \rangle \in X\}$, and*
  - *$\langle s, a \rangle \notin X$ implies $\delta'(s, a) = \bot$.*

- *(The graph underlying) $M[X]$ is a strongly connected component*[1].

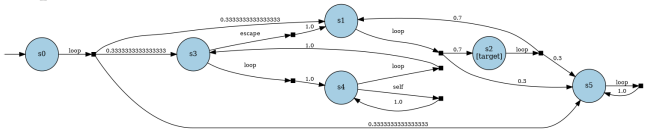
Theorem 1.9. ***Baier and Katoen [2008]***
For any policy π and any state s:

$$\mathsf{Pr}_s^{\pi}(\{\xi \in \mathsf{Paths}^{\pi}(s) \mid \mathsf{inf}(())\xi \text{ is an } EC\}) = 1 \tag{30}$$

That is, the probability that the choices visited infinitely often from any state onwards form an EC is one. In particular, this means that there are still paths that do not eventually end up in an EC, as there are paths that take any loop infinitely often.

End-components can overlap and can contain other end-components. It is often helpful to consider **maximal end components** (MECs): Maximal end components are end components not contained in any end component. MECs cannot overlap. MECs can be detected with an efficient graph algorithm Baier and Katoen [2008].

Example 1.8. *Consider the MDP with six states shown below.*




```
MEC 0: ['s5']
MEC 1: ['s3', 's4']
```

*States s_1 and s_2 have a cycle, but every action has positive probability of escaping to s_5. No policy can keep the process inside $\{s_1, s_2\}$ forever, so this set is **not** an end component.*

State s_5 is absorbing (a self-loop); it forms a trivial MEC.

States s_3 and s_4 each have a "loop" action that transitions to the other state with probability 1. Under the "loop/loop" policy the induced sub-MDP is strongly connected and all transitions remain inside $\{s_3, s_4\}$, so this set is a non-trivial MEC. The fact that s_3 also has an "escape" action to s_1, and s_4 a "self" action, does not matter: a policy exists that stays inside forever, and that is sufficient. These extra actions also create a nested end-component $\{s_4\}$ under the "self" action alone, which is itself contained in the larger MEC $\{s_3, s_4\}$.

We note that sink states with their self-loops are always MECs: We call these *trivial* MECs.

Analogously to the notion of an acyclic MDP, we call an MDP *MEC-free*, if all MECs are trivial.

> **Note**
> Importantly, an MEC-free MDP has MECs, just like an acyclic MDP has cycles.

[1] In a strongly connected component, every state can reach every state

Theorem 1.10. *If the MDP in Theorem 1.7 is MEC-free, then the equations have a unique solution.*

The statement indeed follows directly from Theorem 1.9.

Maximal End-Component collapsingdiscuss! MDPs can be transformed into a MEC-free MDP while preserving *either* min or max reachability probabilities for a fixed target. The transformation is called *MEC collapsing*. We only discuss the MEC collapsing for maximising.

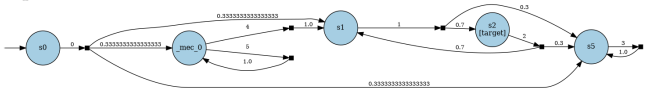
Definition 1.9 (MEC collapsing)**.** *Given an MDP M with MECs $E_1, \ldots, E_k$ and associated state sets $S_1, \ldots S_k$ the collapsed MDP M' is obtained by:*

- *replacing each non-trivial MEC E_i with a single representative state $\hat{E}_i$ carrying the union of labels of all states in S_i,*
- *redirecting every transition into any state in S_i to $\hat{E}_i$, and*
- *For $\hat{E}_i$, add all choices that leave the MEC, i.e., for any $s \in S_i$ with $(s, a) \notin E_i$, we add a new choice $(\hat{E}_i, a_s) = \delta(s, a)$.*
  States outside any non-trivial MEC are unchanged.

The intuition behind the new actions s_a leaving the representative of a MEC is that a policy can choose any (weighted combination) of the original state-action pairs to leave an MEC.

Theorem 1.11. *For any target set T and any state s not inside a non-trivial MEC, the maximal reachability probabilities $\mathsf{Pr}^{\max}(s \models \Diamond T)$ are preserved by MEC collapsing.*

Example 1.9. *We collapse the end components of the MDP from the previous example.*



The non-trivial MEC $\{s_3, s_4\}$ is merged into one representative state. The trivial MEC $\{s_5\}$ and the non-MEC states s_0, s_1, s_2 are preserved unchanged.

1.3 Algorithms for reachability probabilitiesdiscuss!

1.3.1 Policy enumerationdiscuss!

As memoryless deterministic policies suffice, we can compute minimal and maximal reachability probabilities by simply enumerating over all memoryless deterministic policies. We note, however, that this is exponential in the number of states and therefore highly impractical.

1.3.2 Policy iterationdiscuss!

Policy iteration (PI) avoids the exponential cost of policy enumeration. Starting from an initial policy π_0, it alternates between two steps until $\pi_i = \pi_{i+1}$:

- **Policy evaluation**: compute V^{π_i} by solving the linear system of equations induced by π_i.
- **Policy improvement**: set

$$\pi_{i+1}(s) =_a \sum_{s'} \delta(s,a)(s') \cdot V^{\pi_i}(s'), \tag{31}$$

  updating an action only if it yields a strict improvement (replace $\max$ by $\min$ for minimisation).

For maximisation, any initial policy suffices. For minimisation, care is needed: a state $s \in S \setminus S^{min}_{>0}$ has minimum reachability probability 0, but a naive initial policy may assign it a positive value and cause PI to diverge. The fix is to start from a *proper* initial policy: for each $s \in S \setminus S^{min}_{>0}$, pick an action whose support stays within $S \setminus S^{min}_{>0}$. Such an action always exists: $s \notin S^{min}_{>0}$ means exactly that some action has no successor in $S^{min}_{>0}$ (by the fixpoint characterisation of $\Psi_{\min>0}$). Under a proper initial policy, $S \setminus S^{min}_{>0}$ is an absorbing set; policy evaluation naturally assigns those states value 0, and no improvement step ever moves out of that set.

Theorem 1.12. *Policy iteration terminates after finitely many steps and the resulting policy is optimal.*

Example 1.10. *We run policy iteration for minimal reachability probability on Figure 1 with s_2 as the target.*

```
from stormvogel.teaching.policy_iteration import PI, visualise_pi_iterations

pi = PI(mdp_parker, "target", minimize=True)
visualise_pi_iterations(pi)
```

	step 0		*step 1*	
	value	*action*	*value*	*action*
state				
s0	*1*	*b*	*2/3*	*a*
s1	*1*	*a*	*14/15*	*a*
s2	*1*	*a*	*1*	*a*
s3	*0*	*b*	*0*	*b*

PI converges in two steps. The proper initial policy sends s_3 to action b (self-loop within $S \setminus S^{min}_{>0}$), fixing its value to 0. In step 0, s_0 takes action

b (pessimistic: goes to s_1), yielding value 1; improvement then switches s_0 to action a. In step 1 the new scheduler is evaluated to give the exact minimal reachability probabilities, and no further improvement is possible.

Remark 1.3. *Note that the discussion above assumes exact solving of the induced DTMC. Any non-exact solver breaks all guarantees about the correctness of PI.*

1.3.3 Linear programmingdiscuss!

The next algorithm simply reduces the problem of computing reachability probabilities to a linear programming problem.

Short recapdiscuss! Linear programming refers to an optimisation problem (a mathematical program) of a particular form.

Definition 1.10 (Linear programming)**.** *Given n real-valued variables $x_1, \ldots, x_n$ and constants a_{ij}, b_i, c_j, with $1 \leq i \leq m$ and $1 \leq j \leq n$, a linear program is of the form:*

$$\begin{aligned} \textit{Maximize} \quad & c_1 \cdot x_1 + \ldots c_n \cdot x_n \\ & \textit{such that} \\ & a_{11} \cdot x_1 + a_{12} \cdot x_2 + \ldots a_{1n} \cdot x_n \leq b_1 \\ & \ldots \\ & a_{m1} \cdot x_1 + a_{m2} \cdot x_2 + \ldots a_{mn} \cdot x_n \leq b_m \end{aligned}$$

We can support minimisation and $\geq$ by adequately negating constants. Linear programming is supported by a rich theory. Here, it is important to note that linear programming admits polynomial-time solutions and there is very mature (academic and commercial) tool support for solving linear programs.

Reductiondiscuss! We again consider first the minimal reachability probability and then the maximal reachability probability.

Minimal reachability probabilitiesdiscuss! The essence of the Theorem 1.5 is that in every state, we can pick a minimising action. We translate that and say that the value of a state must be smaller than the value induced by any action. It is thus necessarily at most the value of the minimal action. We then maximise the value of every state to ensure that we meet the solution.

Theorem 1.13. *The minimal reachability probability is the unique solution to*

the following LP with variables x_s for $s \in S$.

$$
\begin{aligned}
\textit{maximize} \quad & \sum_{s \in S} x_s & \\
& x_s = 1 & \textit{for all } s \in T \\
& x_s = 0 & \textit{for all } s \in S^{min}_{=0} \\
& x_s \geq \sum_{s'} \delta(s,a)(s') x_{s'} & \textit{for all other } s \textit{ and all } a \in A_{\mathsf{en}}(s)
\end{aligned}
$$

Example 1.11. `LP(sense='maximize', objective=x_s0 + x_s1, constraints=[Eq(x_s2, 1), Eq(x_s3`

Maximal reachability probabilitiesdiscuss! The adaptions are straightforward. What is notable is that the Theorem 1.7 require that we find the smallest solution, which we do by minimising here.

Theorem 1.14. *The maximal reachability probability is the unique solution to the following LP with variables x_s for $s \in S$.*

$$
\begin{aligned}
\textit{minimize} \quad & \sum_{s \in S} x_s & \\
& x_s = 1 & \textit{for all } s \in T \\
& x_s = 0 & \textit{for all } s \in S^{max}_{=0} \\
& x_s \geq \sum_{s'} \delta(s,a)(s') x_{s'} & \textit{for all other } s \textit{ and all } a \in A_{\mathsf{en}}(s)
\end{aligned}
$$

Example 1.12. `LP(sense='minimize', objective=x_s0 + x_s1 + x_s3, constraints=[Eq(x_s2, 1),`

Consequences of the LP reductiondiscuss! From the fact that linear programming can be solved in polynomial time, we get the following important consequence.

Theorem 1.15. *There is a polynomial-time algorithm to compute the value of an MDP.*

In fact, the LP-based approach is the only known polynomial time algorithm for solving MDPs. We note, however, that practically, using LP solvers for MDPs is not very relevant and that most modern LP solvers (except Soplex and Z3) do not provide any formal guarantees about the quality of the solution.

1.3.4 Value Iteration discuss!

Intuitively, the idea of value iteration is to approximate the reachability probabilities by n-step reachability probabilities, for increasing n. As such, the idea is similar to computing the set of reachable states by iteratively adding successor states in a breadth-first manner. However, while in the qualitative case we reason about sets of states, now we must reason about the reachability probability

per state. More formally, we move from the Definition 5.3 $(2^S, \subseteq)$ where 2^S denotes the powerset on S to the lattice $([0,1]^S, \preceq)$ where $\preceq$ denotes pointwise inequality.

Notation
Strictly, 2^S denotes the functions from S to $\{0,1\}$, i.e., functions that determine membership for every state. Likewise, $[0,1]^S$ denotes the functions from S to $[0,1]$, i.e., functions that assign a probability to every state. Assuming a total order on states, $[0,1]^S$ can be interpreted as $|S|$-dimensional vectors, which is also convenient in examples.

To formally describe the iterative update of the reachability probabilities, we use *Bellman operators*. Where *Bellman equations* describe the optimal solution, the Bellman operators are formal operators that describe an update of values. We first present the operator for the minimal reachability probabilities, and then for maximal reachability probabilities.

Minimal reachability probabilities discuss! Recall the Theorem 1.5.

Definition 1.11 (Bellman operator (MinReachProb))**.** *The Bellman operator for minimal reachability probabilities and for a fixed MDP is a mapping* $\Phi\colon [0,1]^S \to [0,1]^S$ *s.t.*

$$\Phi(F)(s) = \begin{cases} 1 & \text{if } s \in T, \\ 0 & \text{if } s \in S^{min}_{=0}, \\ \min_{a \in A_{\mathsf{en}}(s)} \sum_{s' \in S} \delta(s,a,s') F(s') & \text{otherwise.} \end{cases} \tag{32}$$

Example 1.13. *Recall the Bellman equations from Example 1.5. We now show the Bellman operator:*

<IPython.core.display.Math object>

Let us execute the Bellman operator, maybe first on the bottom element of the lattice, which assigns zero to every state.

<IPython.core.display.Math object>

Indeed, the probabilities we see here are the zero-step reachability probabilities to the target states.

If we apply the Bellman operator once more, we get the one-step minimal reachability probabilities:

<IPython.core.display.Math object>

And after one more application, the two-step reachability probabilities.

<IPython.core.display.Math object>

After looking at a concrete example, let us consider a more generic setting. $\Phi(\mathbf{0})$ yields the indicator function for T, $\not\Vdash_T$. $\Phi(\not\Vdash_T)$ yields the one-step minimal reachability probabilities to reach T. By simple substitution, we have $\Phi(\Phi(\mathbf{0})) = \Phi(\not\Vdash_T)$. Likewise, $\Phi(\Phi(\Phi(\mathbf{0})))$, denoted $\Phi^3(\mathbf{0})$ yields the two-step minimal reachability probabilities and indeed, $\Phi^{n+1}(\mathbf{0})$ denotes the n-step minimal reachability probabilities.

Lemma 1.16. *The Definition 1.11 Φ is monotonic and ω-continuous.*

Theorem 1.17 (MinReachProb is a fixpoint)**.** *Let Φ be the Definition 1.11. It holds that:*

- *The value is a unique fixpoint, i.e.:*

$$\mathsf{lfp}.\Phi = \mathsf{gfp}.\Phi = V^{\min}, \tag{33}$$

  and in particular also

$$\Phi(V^{\min}) = V^{\min}. \tag{34}$$

- $\Phi^n(\mathbf{0}) \leq V^{\min}$ *for all n and* $\Phi^n(\mathbf{1}) \geq V^{\min}$ *for all n.*

Value iteration algorithmdiscuss! The essence of value iteration is thus to start with $F \leftarrow \mathbf{0}$ and repeatedly apply $F \leftarrow \Phi(F)$ until some termination condition is met. The problem is that we may not reach the fixpoint (the n-step reachability may always be smaller than the unbounded reachability). What we realistically hope to achieve is

$$|F - V^*| \leq \epsilon \quad \text{(pointwise)} \tag{35}$$

.

Example 1.14. *Consider the Bellman operator from Example 1.13. We run value iteration using this operator.*

iteration	*0*	*1*	*2*	*3*	*4*	*5*	*6*	*7*
s0	*0.0*	*0.0*	*0.4*	*0.60*	*0.65*	*0.6625*	*0.66563*	*0.66641*
s1	*0.0*	*0.4*	*0.6*	*0.74*	*0.83*	*0.8800*	*0.90625*	*0.91969*
s2	*1.0*	*1.0*	*1.0*	*1.00*	*1.00*	*1.0000*	*1.00000*	*1.00000*
s3	*0.0*	*0.0*	*0.0*	*0.00*	*0.00*	*0.0000*	*0.00000*	*0.00000*

As the fixpoint is unique, we could alternatively start elsewhere and still converge against the optimum.

iteration	*0*	*1*	*2*	*3*	*4*	*5*	*6*	*7*
s0	*1.0*	*0.75*	*0.6875*	*0.67188*	*0.66797*	*0.66699*	*0.66675*	*0.66669*
s1	*1.0*	*1.00*	*0.9750*	*0.95625*	*0.94531*	*0.93945*	*0.93643*	*0.93489*
s2	*1.0*	*1.00*	*1.0000*	*1.00000*	*1.00000*	*1.00000*	*1.00000*	*1.00000*
s3	*0.0*	*0.00*	*0.0000*	*0.00000*	*0.00000*	*0.00000*	*0.00000*	*0.00000*

A natural stopping criterion for VI is to halt when successive iterates differ by at most ε pointwise, i.e.\ $\|\Phi^{n+1}(\mathbf{0}) - \Phi^n(\mathbf{0})\|_\infty \leq \varepsilon$. Unfortunately, this *local* near-convergence does not imply that the current iterate is within ε of $V^{\min}$. For MDPs with end components the gap between $\Phi^n(\mathbf{0})$ and $V^{\min}$ can remain arbitrarily large even when successive iterates are indistinguishable Haddad and Monmege [2018]. Concretely, the lower sequence $\Phi^n(\mathbf{0})$ may converge to a value strictly below $V^{\min}$ at every finite step, with the true value only reached in the limit. Stopping early therefore yields a systematic underestimate with no bound on the error.

Interval iteration algorithmdiscuss! Rather than relying on local near-convergence, *interval iteration* (IVI) runs two VI sequences simultaneously: one from $\mathbf{0}$ (lower bound) and one from $\mathbf{1}$ (upper bound). Because the fixpoint is unique, both sequences converge to $V^{\min}$, and the gap between them shrinks monotonically. When lower and upper agree within tolerance ε, we have a guaranteed ε-approximation.

Example 1.15. *We run interval iteration on the Parker MDP for minimal reachability probabilities.*

iteration	*0*	*1*	*2*	*3*	*4*	*5*	*6*	*7*
s0	*(0.0, 1.0)*	*(0.0, 0.75)*	*(0.4, 0.6875)*	*(0.6, 0.67188)*	*(0.65, 0.66797)*	*(0.6625, 0.66699)*	*(0.66563, 0.66675)*	*(0.66641, 0.66669)*
s1	*(0.0, 1.0)*	*(0.4, 1.0)*	*(0.6, 0.975)*	*(0.74, 0.95625)*	*(0.83, 0.94531)*	*(0.88, 0.93945)*	*(0.90625, 0.93643)*	*(0.91969, 0.93489)*
s2	*(1.0, 1.0)*	*(1.0, 1.0)*	*(1.0, 1.0)*	*(1.0, 1.0)*	*(1.0, 1.0)*	*(1.0, 1.0)*	*(1.0, 1.0)*	*(1.0, 1.0)*
s3	*(0.0, 0.0)*	*(0.0, 0.0)*	*(0.0, 0.0)*	*(0.0, 0.0)*	*(0.0, 0.0)*	*(0.0, 0.0)*	*(0.0, 0.0)*	*(0.0, 0.0)*

Each cell shows a `(lower, upper)` *pair. The bounds tighten with each step and converge to the same values as standard VI.*

Maximal reachability probabilities discuss!

Definition 1.12 (Bellman operator (MaxReachProb))**.** *The Bellman operator for maximal reachability probabilities (MaxReachProb) and for a fixed MDP is a mapping* $\Phi\colon [0,1]^S \to [0,1]^S$ *s.t.*

$$\Phi(F)(s) = \begin{cases} 1 & \text{if } s \in T, \\ 0 & \text{if } s \in S^{max}_{=0}, \\ \max_{a \in A_{\text{en}}(s)} \sum_{s' \in S} \delta(s,a,s') F(s') & \text{otherwise.} \end{cases} \tag{36}$$

The intuition behind the Bellman operator is the same as for the Bellman operator for minimal reachability probabilities.

Lemma 1.18. *The Definition 1.12 Φ is monotonic and ω-continuous.*

Theorem 1.19 (MaxReachProb is a fixpoint)**.** *Let Φ be the Definition 1.12 for an MDP M. It holds that:*

- *The value is the least fixpoint, i.e.:*

$$\mathsf{lfp}.\Phi = V^{\max}, \tag{37}$$

  and in particular also

$$\Phi(V^{\max}) = V^{\max}. \tag{38}$$

- *If M is MEC-free, then there is a unique fixed point, i.e.,*

$$\mathsf{lfp}.\Phi = V^{\max} = \mathsf{gfp}.\Phi. \tag{39}$$

- $\Phi^n(\mathbf{0}) \leq V^{\max}$ *for all n.*

The application of value iteration is therefore similar to before. Iterating from $\mathbf{0}$ yields a correct result. In contrast to the minimal reachability probability, it is important how we initialise the value iteration as there are multiple fixpoints. The simple interval iteration therefore also does not converge and one must ensure that we converge against the least fixed point. The standard way to ensure this is by eliminating maximal end components.

Example 1.16. *We run interval iteration on the MDP from the MEC example (see Maximal end componentsdiscuss!), computing the maximum probability of reaching s_2 (labeled target).*

iteration	*0*	*1*	*2*	*3*	*4*	*5*	*6*	*7*
s0	*(0.0, 1.0)*	*(0.0, 0.66667)*	*(0.23333, 0.56667)*	*(0.46667, 0.56667)*	*(0.46667, 0.56667)*	*(0.46667, 0.56667)*	*(0.46667, 0.56667)*	*(0.46667, 0.56667)*
s1	*(0.0, 1.0)*	*(0.7, 0.7)*	*(0.7, 0.7)*	*(0.7, 0.7)*	*(0.7, 0.7)*	*(0.7, 0.7)*	*(0.7, 0.7)*	*(0.7, 0.7)*
s2	*(1.0, 1.0)*	*(1.0, 1.0)*	*(1.0, 1.0)*	*(1.0, 1.0)*	*(1.0, 1.0)*	*(1.0, 1.0)*	*(1.0, 1.0)*	*(1.0, 1.0)*
s3	*(0.0, 1.0)*	*(0.0, 1.0)*	*(0.7, 1.0)*	*(0.7, 1.0)*	*(0.7, 1.0)*	*(0.7, 1.0)*	*(0.7, 1.0)*	*(0.7, 1.0)*
s4	*(0.0, 1.0)*	*(0.0, 1.0)*	*(0.0, 1.0)*	*(0.7, 1.0)*	*(0.7, 1.0)*	*(0.7, 1.0)*	*(0.7, 1.0)*	*(0.7, 1.0)*
s5	*(0.0, 0.0)*	*(0.0, 0.0)*	*(0.0, 0.0)*	*(0.0, 0.0)*	*(0.0, 0.0)*	*(0.0, 0.0)*	*(0.0, 0.0)*	*(0.0, 0.0)*

The lower bounds for s_3 and s_4 converge to 0.7 (the true value, reached via the escape action), but the upper bounds remain stuck at 1.0. They support each

other: the Bellman update for s_3 sees $\max(\textit{escape} \to 0.7,\ \textit{loop} \to V(s_4) = 1) = 1$*, and symmetrically for s_4. The gap never closes.*

After MEC collapsing (with the representative self-loop removed so the upper bound can descend), the equations have a unique fixed point and IVI converges:

iteration	*0*	*1*	*2*	*3*	*4*	*5*
_mec_0	*(0.0, 1.0)*	*(0.0, 1.0)*	*(0.7, 0.7)*	*(0.7, 0.7)*	*(0.7, 0.7)*	*(0.7, 0.7)*
s0	*(0.0, 1.0)*	*(0.0, 0.66667)*	*(0.23333, 0.56667)*	*(0.46667, 0.46667)*	*(0.46667, 0.46667)*	*(0.46667, 0.46667)*
s1	*(0.0, 1.0)*	*(0.7, 0.7)*	*(0.7, 0.7)*	*(0.7, 0.7)*	*(0.7, 0.7)*	*(0.7, 0.7)*
s2	*(1.0, 1.0)*	*(1.0, 1.0)*	*(1.0, 1.0)*	*(1.0, 1.0)*	*(1.0, 1.0)*	*(1.0, 1.0)*
s5	*(0.0, 0.0)*	*(0.0, 0.0)*	*(0.0, 0.0)*	*(0.0, 0.0)*	*(0.0, 0.0)*	*(0.0, 0.0)*

Both bounds meet by iteration 4.

1.3.5 Dynamic programmingdiscuss!

We briefly discuss a dynamic programming approach for acyclic MDPs. Note that by definition, acyclic MDPs are also MEC-free. We therefore only discuss the minimal reachability probability case here; the maximal reachability probabilities can be computed exactly analogously. First, let us note that on acyclic models, VI terminates with the exact solution after h steps, where h is the length of the longest path to a sink state (without looping in the sink states). This yields a quadratic run time for value iteration. By using dynamic programming, we can reduce the quadratic run time to a linear run time.

In particular, if we sort the MDP topologically (i.e., by ascending distance to the target), the following dynamic programming scheme yields the correct solution.

Algorithm
Inputs A Bellman operator and topologically sorted states $\mathsf{sorted}(S)$.
Outputs The least fixed point of that operator.
for every $s \in \mathsf{sorted}(S)$:
$V(s) \leftarrow \Phi(V)(s)$

Theorem 1.20. *Computing (minimal or maximal) reachability probabilities in acyclic MDPs can be done in linear time (in the size of the MDP).*

1.3.6 Combining methodsdiscuss!

Beyond the three main families of algorithms, one can combine different algorithms. The most relevant one is VI2PI: First VI, then PI. This yields the speed of VI (?!) and ensures an exact result with PI.

1.4 From reachability to temporal properties (and back)discuss!

So far, our exposition focussed exclusively on reachability probabilities. In particular, we categorised paths as good or bad, solely depending on whether these paths visit a particular state. In this section, we support more general properties on paths.

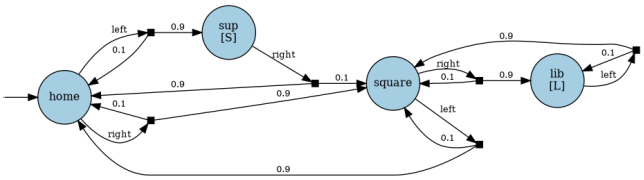
1.4.1 DFA propertiesdiscuss!

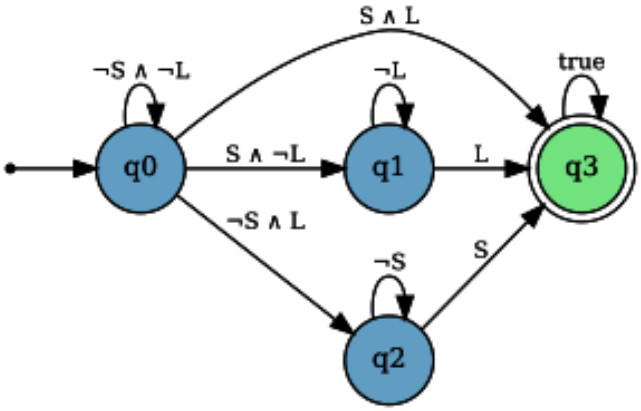
In a first step, we define paths that are accepted by a DFA. More precisely, to support such properties, we label states with propositions.

Definition 1.13. *An MDP with states S can be labelled by defining:*

- *a finite set of atomic propositions* AP*,*
- *a labelling function* $L\colon S \to 2^{\mathsf{AP}}$*.*

Example 1.17. *We consider the following small MDP.*





Example 1.18.

To formalise what we want, we first lift paths to traces over these executions:

The AP-trace of a path $\xi = s_0 a_0 s_1 a_1 \dots$ omits the actions and lifts states to the labels:

$$\mathsf{tr}(\xi) = L(s_0)L(s_1)\cdots \in \left(2^{\mathsf{AP}}\right)^* \tag{40}$$

. The set of all AP-traces is called Tr.

In a Markov chain, we can easily lift the probability of a path to the probability of a trace:

$$\mathsf{Pr}(\tau) = \sum_{\xi \in \mathsf{Paths}, \mathsf{tr}(\xi) = \tau} \mathsf{Pr}(\xi). \tag{41}$$

Given a set of traces $X \subseteq \mathsf{Tr}$, we can thus also define $\mathsf{Pr}(X)$ as the probability of generating a path ξ s.t. $\mathsf{tr}(\xi) \in X$.

We consider properties that can be described by deterministic finite automata. Given an MDP with atomic propositions AP, a DFA with alphabet 2^{AP} describes a *DFA* property.

Definition 1.14. *Given a Markov chain with atomic propositions* AP *and a DFA* $\mathcal{A}$ *with alphabet* 2^{AP}*: Let* $\mathcal{L} \subseteq \left(2^{\mathsf{AP}}\right)^*$ *be the Definition 5.2 of* $\mathcal{A}$*. The satisfaction property for the DFA property* $\mathsf{Pr}(s \models \mathcal{A})$ *is given as:*

$$\mathsf{Pr}(s \models \mathcal{A}) = \mathsf{Pr}(\{\xi \in \mathsf{Paths}(s) \mid \mathsf{tr}(\xi) \in \mathcal{L}\}). \tag{42}$$

For MDPs, we then lift to minimal and maximal probabilities analogously to reachability probabilities. Note however, that memoryless policies are not sufficient:

Lemma 1.21. *There exist MDPs and DFAs such that:*

1.
$$\sup_{\pi \in \Pi} \mathsf{Pr}^\pi(\Diamond T) > \max_{\pi \in \Pi_{md}} \mathsf{Pr}^\pi(\Diamond T) \tag{43}$$

2.
$$\inf_{\pi \in \Pi} \mathsf{Pr}^\pi(\Diamond T) < \min_{\pi \in \Pi_{md}} \mathsf{Pr}^\pi(\Diamond T) \tag{44}$$

Reduction to reachability **discuss!** We can compute the (minimal/maximal) probability to satisfy a DFA property by a product construction.

Definition 1.15. *Let* $M = \langle S, A, \delta_M$ *be an MDP with initial state* s_{init}*, atomic propositions* AP *and labelling function* L*, and let* $\mathcal{A} = \langle Q, \mathsf{AP}, q_0, \delta_\mathcal{A}, \mathsf{Acc}\rangle$ *be a DFA. We define the product MDP*

$$M \otimes \mathcal{A} = \langle S \times Q, A, \delta' \rangle \tag{45}$$

with initial state $\langle s_{init}, \delta_\mathcal{A}(q_0, L(s_{init}))\rangle$ *and a transition relation such that*

$$\delta'(\langle s, q\rangle, a)(\langle s', q'\rangle) = \delta_M(s, a)(s') \cdot [\delta_\mathcal{A}(q, L(s')) = q'] \tag{46}$$

.

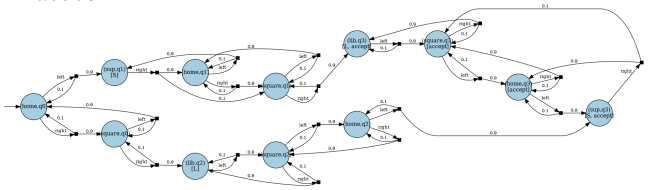
Specifically, the state space contains the MDP state and the DFA state. In every transition, we update the DFA state by reading the label of the new MDP state.

In the product, we can then define target states as states that correspond to an accepting state in the DFA.

Theorem 1.22. *For any MDP M with states S, DFA property $\mathcal{A}$ with accepting states* Acc*, and* $\mathrm{opt} \in \{\min, \max\}$

$$\Pr_M^{\mathrm{opt}}(s \models \mathcal{A}) = \Pr_{M \otimes \mathcal{A}}^{\mathrm{opt}}(\Diamond\{\langle s, q\rangle \mid q \in \mathsf{Acc}\}). \tag{47}$$

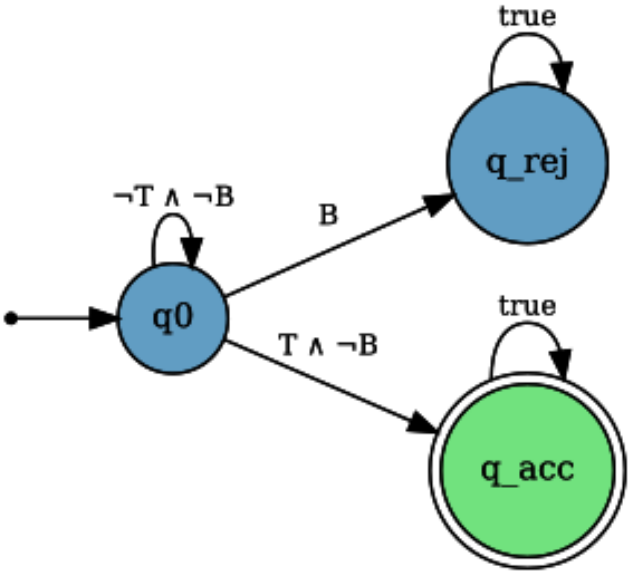
Example 1.19. *The following visualises the product MDP for the DFA and the MDP above.*



Reach-avoid and step-bounded propertiesdiscuss! We highlight two simple instances of DFA properties. Algorithmically, one typically avoids building the full product for these properties, but semantically, it is adequate to think about the product.

Reach-avoid propertiesdiscuss! In a *reach-avoid* property, one aims to reach the target states T while avoiding another set of bad states B. For simplicity, let us assume that reach and avoid states are disjoint. A reach-avoid property can be simply captured by a 3-state DFA:

- The initial state,
- an absorbing, accepting state reached after seeing a target in the MDP,
- an absorbing, not accepting state, reached after seeing an avoid state in the MDP. In particular, exactly the paths that visit a reach state, before reaching an avoid state will be accepted by the DFA. Below, we exemplify the construction. For the sake of completeness, we assume that a path that visits target states and bad states simultaneously should not be accepted.



Example 1.20.

Step-bounded propertiesdiscuss! A *step-bounded* property with horizon h asks for reaching a target state within h steps. A DFA for this property consists of $h+2$ states: We start in a state encoding that has made zero steps so far. If a target state is visited in the MDP, we go to an absorbing and accepting state. Otherwise, we go from the state encoding i steps so far to $i+1$ steps so far. The state encoding $h+1$ steps is absorbing and should be interpreted as having exceeded the horizon. Step-bounded properties can be generalised to cost-bounded properties.

1.4.2 Büchi propertiesdiscuss!

Reachability and DFA properties consider paths up to some (indefinite) horizon. Instead, in Büchi properties, we consider infinite paths. Essentially, Büchi properties ask that a set of (target) states are visited infinitely often along a path.

Remark 1.4. *To avoid confusion, we call the target states Büchi states in this subsection.*

Definition 1.16. *Given a set of Büchi states T, an infinite path $s_0 s_1 \ldots$ satisfies a Büchi property $\square\lozenge T$ iff there are infinitely many i such that $s_i \in T$.*

As such, Büchi properties allow expressing that one can reach some good state and that one is able to visit from that state another good state. They help to state that there is not a bounded number of recharge actions, etc. Büchi properties are a specific type of so-called limit properties, where the acceptance of a path is really determined by whatever happens infinitely often. In finite

systems, this specifically means that we can concentrate our analysis on end-components.

Theorem 1.23. *For any π:*

$$\mathsf{Pr}^{\pi}(\xi \in \mathsf{Paths} \mid \underbrace{\mathsf{inf}(\xi)}_{\substack{\text{choices visited} \\ \text{inf often}}} \text{ is an EC }) = 1 \tag{48}$$

The proof first observes that for any infinite path in a finite MDP, there must be at least one choice which is visited infinitely often. Every successor state of that choice must also be visited infinitely often, and thus in those states, also a choice is visited infinitely often. This ascending set then stabilises with a strongly connected component where a policy can stay forever, i.e., an EC.

Definition 1.17. *An EC satisfies a Büchi property φ if it contains a target state.*

We justify this wording by remarking that a policy can choose to visit any state in an EC infinitely often. In particular, such a policy can thus satisfy φ.

Theorem 1.24.

- *Let U_φ be the union of all S' in an EC that satisfies the Büchi property φ.*
- *Let V_φ be the union of all S' in an EC that does not satisfy the Büchi property φ. Then:*

1. $\mathsf{Pr}^{\max}(\varphi) = \mathsf{Pr}^{\max}(\Diamond U_\varphi)$
2. $\mathsf{Pr}^{\min}(\varphi) = 1 - \mathsf{Pr}^{\max}(\Diamond V_\varphi)$.

The theorem above gives rise to a (naive) algorithm: Compute all ECs, classify each EC as satisfying or not satisfying, and then solve a reachability probability problem. Note that such an algorithm is not efficient, as there are exponentially many ECs. However, careful graph algorithms with a nested fixpoint operation avoid the necessity to precompute all ECs, while in spirit identifying ECs.

> **Attention**
> Skipped for now.

1.4.3 Towards LTL and ω-regulardiscuss!

> **Attention**
> Skipped for now.

1.5 Rewardsdiscuss!

We now extend MDPs with rewards. Rewards are commonly used to describe resource usage (time, energy, ...). They can also be used to define desired and undesired actions or states.

Definition 1.18 (MDP with rewards)**.** *An MDP with states S may have*

- *state rewards* $r\colon S \to \mathbb{Q}_{\geq 0}$[2] *or,*
- *choice rewards* $r\colon S \times A \to \mathbb{Q}_{\geq 0}$*.*
- *transition rewards* $r\colon S \times A \times S \to \mathbb{Q}_{\geq 0}$*.*

We consider choice rewards in the lecture notes (which coincides with state rewards obtained upon exiting a state in a Markov chain).

> **Cost vs reward.**
> Rewards and costs are used mostly interchangeably. As a general rule of thumb, the word reward is preferred when one tries to maximise (or bound from below) the quantity, whereas cost is preferred when one tries to minimise (or bound from above) the quantity. In these lecture notes, we stick to rewards.

Given a path $\xi = s_0 a_0 s_1 \ldots$, the reward of ξ is defined as

$$\mathsf{rew}(\xi) = \sum_i = r(s_i, a_i) \tag{49}$$

.

1.5.1 Expected reachability rewardsdiscuss!

So far, we have partitioned infinite (or finite) paths into good and bad paths. We've then computed policies that optimise the probability mass of good paths. In this section, we use the annotation with rewards to weigh every path and optimise the *expected* reward, i.e., the weighted sum of the probability and the weight of every path.

Consider the reachability reward for infinite paths:

$$\mathsf{rew}_{\Diamond T}(s_0 s_1 s_2 \ldots) = \begin{cases} \mathsf{rew}(s_0 a_0 \ldots s_n) & \text{if } s_i \notin T \text{ for all } i < n \text{ and } s_n \in T \\ \infty & \text{otherwise.} \end{cases} \tag{50}$$

[2]Mixing positive/negative rewards requires more cornercases and is therefore not part of this material.

Expected reachability rewards in Markov chains.discuss!

Definition 1.19. *In Markov chains, the expected reward until reaching T, $\mathbb{E}(\Diamond T)$ is defined as:*

$$\sum_{r=0}^{\infty} r \cdot \mathsf{Pr}(\{\xi \in \Diamond T \mid \mathsf{rew}_{\Diamond T}(\xi) = r\}). \tag{51}$$

Lemma 1.25. *The expected reward is finite iff the the target states are reached with probability one.*

For expected rewards, a standard linear equation system can be formulated:

Theorem 1.26. *Given a Markov chain with target states T. Assume the expected reward until reaching T is finite. Then the expected reward until reaching T is the unique solution to the linear equation system over real-valued variables x_s, $s \in S$:*

$$x_s = \begin{cases} 0 & \text{if } s \in T \\ r(s) + \sum P(s)(s') \cdot x_{s'} & \text{otherwise} \end{cases} \tag{52}$$

Maximal expected reachability rewardsdiscuss! For MDPs, we can study reachability rewards under a minimising or a maximising policy. We write $\mathbb{E}^{\pi}(\Diamond T)$ for the expected reward in the Markov chain induced by π. We start with maximal expected rewards. As before, we definite the maximal expected reward as

$$\mathbb{E}^{\max}(\Diamond T) = \sup_{\pi \in \Pi} \mathbb{E}^{\pi}(\Diamond T). \tag{53}$$

Below, we follow the exposure in Chatterjee et al. [2025]. In particular, we define another Bellman operator.

Definition 1.20. *Let* $\oplus_{\mathbb{E}}^{\max} \colon \mathbb{R}^S_{\infty,\geq 0} \to \mathbb{R}^S_{\infty,\geq 0}$

$$\oplus_{\mathbb{E}}^{\max}(F)(s) = \begin{cases} 0 & \text{if } s \in T, \\ \max_{a \in A_{\mathsf{en}}(s)} \mathrm{rew}(s,a) + \sum_{s'} \delta(s,a,s') \cdot F(s') & \text{if } s \in S^{min}_{>0}, \\ \infty & \text{otherwise} \end{cases} \tag{54}$$

In the operator, we see that we explicitly assign infinite reward whenever there is a possibility for a policy to reach the target with probability zero.

Theorem 1.27.

$$\mathsf{lfp}.\oplus_{\mathbb{E}}^{\max} = \mathbb{E}^{\max}(\Diamond T). \tag{55}$$

It also follows that there is a memoryless deterministic policy that optimises the expected reward. We can compute the expected reward by value iteration or policy iteration.

Attention
Skipped for now.

Minimal expected reachability rewardsdiscuss!

Reachability probabilities as expected rewardsdiscuss! Expected reachability rewards can express reachability probabilities.

1.5.2 Expected discounted total rewardsdiscuss!

Discounted rewards in Markov chainsdiscuss! In a lot of MDP literature on planning (or control) problems, infinite horizon (total rewards) are studied. That is, there are no target states and the reward along a path may diverge. A common mathematical trick is to exponentially discount reward with a factor $\gamma \in (0, 1)$.

The discounted reward of a (possibly infinite) path is then defined as:

$$\mathsf{disc}_\gamma(s_0 a_0 s_1 a_1 \dots) = \sum_i \gamma^i r(s_i) \tag{56}$$

. We note that this sum always exists (and is finite) as $r(s)$ is bounded from above.

The discounted expected reward for a Markov chain is then defined as:

Definition 1.21 (Discounted reward)**.** *For Markov chain M and $\gamma \in (0, 1)$, we define the expected discounted reward $\mathbb{E}_M(s \models \mathsf{tot}_\gamma)$ as*

$$\mathbb{E}_M(s \models \mathsf{tot}_\gamma) = \int_{\xi \in \mathsf{Paths}(s)} \mathsf{Pr}(\xi) \cdot \mathsf{disc}_\gamma(\xi). \tag{57}$$

As disc_γ is bounded from above, the integral exists.

Discounting the future is well-motivated in finance, where immediate gains are worth more than later gains. It is also commonly motivated to be useful to prioritize nearby good actions over future actions (as a model may not predict the long-term future well). In general, it is fair to say that with discounted rewards, one may find useful policies for a planning problem (where the goal is to find the policy). However, the correct interpretation of a discounted reward or any guarantee about it is hard and subject to modelling decisions.

Attention
Content missing: Express total discounted reward as reachbility reward

Optimal discounted total rewards in MDPsdiscuss! We can now define $\mathbb{E}^{\max}(s \models \mathsf{tot}_\gamma)$ and $\mathbb{E}^{\min}(s \models \mathsf{tot}_\gamma)$ analogously to $\mathsf{Pr}^{\min}$ and $\mathsf{Pr}^{\max}$. As before, memoryless deterministic policies suffice.

Bellman operatordiscuss! Likewise, we can define Bellman equations and the operators. For conciseness, we only define the operator:

Definition 1.22 (Bellman operator (MaxDiscReward/MinDiscReward))**.** *The Bellman operator for maximal discounted total reward and for a fixed MDP is a mapping* $\Phi^{\max}_{\mathsf{disc}_\gamma} : [0,\infty)^S \to [0,\infty)^S$ *s.t.*

$$\Phi^{\max}_{\mathsf{disc}_\gamma}(F)(s) = \max_{a \in A_{\mathsf{en}}(s)} r(s,a) + \gamma \cdot \sum_{s' \in S} \delta(s,a,s')F(s'). \tag{58}$$

The Bellman operator for minimal discounted total reward is defined analogously.

We observe that this Bellman operator is a contraction operator and that thus, Banach's fixpoint theorem applies. In particular:

Theorem 1.28. *For the Bellman operators for discounted total reward the following holds:*

$$\mathsf{lfp}.\Phi^{\max}_{\mathsf{disc}_\gamma} = \mathsf{gfp}.\Phi^{\max}_{\mathsf{disc}_\gamma} = \mathbb{E}^{\max}(\mathsf{tot}_\gamma) \tag{59}$$

and

$$\mathsf{lfp}.\Phi^{\min}_{\mathsf{disc}_\gamma} = \mathsf{gfp}.\Phi^{\min}_{\mathsf{disc}_\gamma} = \mathbb{E}^{\min}(\mathsf{tot}_\gamma). \tag{60}$$

Algorithmsdiscuss! PI, VI, and LP all apply. Furthermore, PI and VI can compute ε-precise results in polynomial time. While using a discount factor towards one converges against the undiscounted total reward, the performance of VI and PI quickly degrades with higher discount factors.

Remark 1.5. *The undiscounted total reward can also be defined, but will be infinite if there exists a reachable MEC with non-zero reward. If no such MEC exists, then this property can be rewritten as reachability reward in a modified MDP with a fresh target state. See also*

1.5.3 Cost-bounded reachabilitydiscuss!

> **Attention**
> All content here is still missing.

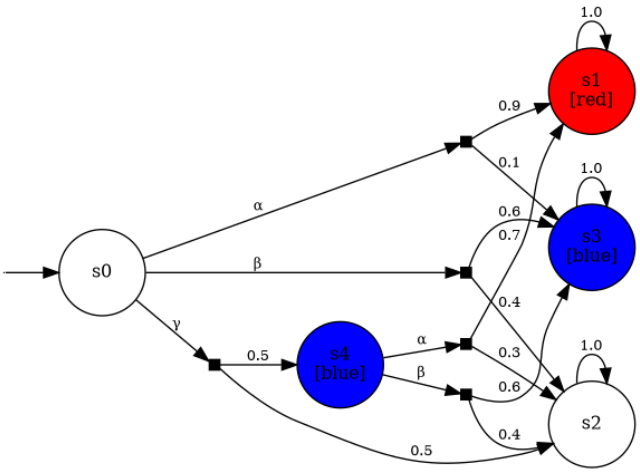
2 Multiobjective Model Checking and other Trade-offs

In the previous chapter, we discussed MDP model checking where a reachability probability or a reward had to exceed a threshold or remain below a threshold. To this end, we considered policies that maximised or minimised the probability or reward. In this chapter, we look at richer specifications where we cannot simply optimize one reachability probability or reward.

2.1 Multiobjective Model Checkingdiscuss!

We study MDPs where there are multiple target sets that we aim to reach.

Example 2.1. *Consider the MDP below. If the goal is to only optimize reaching the red states, taking α in s_0 is optimal. If the goal is to reach the blue states, taking β in s_0 is optimal. However, if both goals are equally relevant, then taking γ in s_0 may be preferred.*



With multiobjective model checking queries, we make the notion of achieving multiple goals concrete.

2.1.1 What are Multiobjective Model Checking queries?discuss!

Throughout this chapter, we fix an MDP M with n target sets $T_1, \ldots, T_n$, $T_i \subseteq S$. The central multiobjective model checking problem is the following:

Definition 2.1 (Multiobjective Model Checking Achievability Query)**.** *Fix an MDP M with m target sets $T_1, \ldots, T_m$, $T_i \subseteq S$. A policy π achieves a point $\vec{\lambda}$, if*

$$\bigwedge_{i \leq m} \mathsf{Pr}_\pi(\Diamond T_i) \geq \lambda_i. \tag{61}$$

We call π a witnessing policy for $\vec{\lambda}$. A point $\vec{\lambda} \in \mathbb{R}^m$ is achievable if there exists a witnessing policy, i.e., $\vec{\lambda}$ is achievable if

$$\exists \pi \in \Pi. \bigwedge_{i \leq m} \mathsf{Pr}_\pi(\Diamond T_i) \geq \lambda_i. \tag{62}$$

We denote the set of achievable points with $\mathsf{Ach}_M(T_1, \ldots, T_m)$.

We simply write Ach whenever M and $T_1, \ldots, T_m$ is clear from the context.

Example 2.2. *Continuing the example above, the policy π with $\pi(s_0) = \gamma$ and $\pi(s_4) = \alpha$ witnesses the achievability of $(0.5, 0.35)$. While we can reach the red states with probability 0.9 and the blue states with probability 0.6, the point $(0.9, 0.6)$ is not achievable.*

Note that the literature considers a much wider variation of combinations of different objectives, including the combination of minimal and maximal reachability probabilities, the extension to rewards, cost-bounded reachability probabilities, LTL formulas, etc., as well as their combinations.

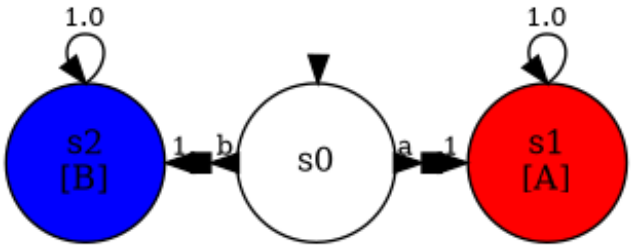
2.1.2 On witnessing policiesdiscuss!

First, deterministic policies are not sufficient:

Lemma 2.1. *There exist MDPs with 2 targets and achievable points such that every witness is randomizing, i.e., where no deterministic witnessing policy exists.*

We provide a proof by the following example.

Example 2.3. *Consider the MDP below and the point $\lambda = (0.5, 0.5)$.*

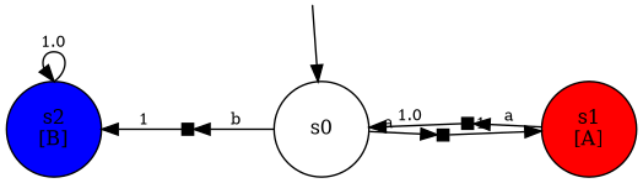


A policy that picks action a and action b both with probability half achieves exactly $(0.5, 0.5)$. Thus, λ is achievable. However, no deterministic policy achieves λ. Specifically, in the MDP, there are two deterministic policies, that either pick a or b in the initial state. The first policy achieves point $(1, 0)$, the second policy achieves $(0, 1)$. Thus, both do not achieve λ.

Second, memoryless policies are not sufficient:

Lemma 2.2. *There exist MDPs with 2 targets and achievable points such that every witness requires memory, i.e., where no memoryless witnessing policy exists.*

Example 2.4. *Consider the MDP below and the point* $\lambda = (1, 1)$.



Specifically, a witnessing policy visits first the red state s_1 *before visiting the blue state* s_2*. In particular, the policy changes its mode once we enter* s_1*: The policy now only wants to optimize reaching the blue state* s_2*. As we formalise Definition 2.4, the policy must thus track which target sets have already been visited.*

However, it suffices to consider randomising finite-memory policies.

Definition 2.2. *A randomising finite-memory policy is a policy that can be represented as an annotated finite state machine, i.e., by a tuple* $\langle Q, \alpha, \delta, q_0 \rangle$ *with*

- *a finite set of nodes* Q *and initial node* q_0*,*
- *an annotation* $\alpha \colon Q \times S \to \mathsf{Distr}(A)$*,*
- *transition relation* $Q \times S \to Q$*.*

The finite state machine defines policy π *with*

$$\pi(s_0 \dots s_n) = \alpha(\delta(q_0, s_1 \dots s_{n-1}), s_n). \tag{63}$$

Theorem 2.3. *A point* $\vec{\lambda}$ *is achievable iff there exists a randomising finite-memory policy that achieves* $\vec{\lambda}$*.*

2.1.3 On achievable pointsdiscuss!

We can look at the geometric properties of Ach. We use notions such as convex and downward closed sets from Geometrydiscuss!.

Theorem 2.4. *The set* Ach *is convex and downward-closed.*

Downward closure follows immediately from the definition of achievability. The convex closure is a bit more interesting. Intuitively, the idea is that if we pick two achievable points $\vec{p}_1, \vec{p}_2$, we can take two witnessing policies π_1, π_2. If we now implement a protocol where we flip a coin with probability c, we play like policy π_1 and with probability $1 - c$, we play like π_2. One needs some care to correctly define such convex combinations of policies. Consider, e.g., that π_1 stays forever in an MEC and π_2 leaves that MEC. A naive realization of a convex combination of both policies would always leave the MEC. A thorough treatment of this topic can be found, e.g., in Quatmann [2023].

Definition 2.3. *A point $\vec{p} \in \mathbb{R}^m$ is Pareto-optimal for targets $T_1, \ldots, T_m$ iff for all $\vec{q} \in \mathbb{R}^m$:*

- *$\vec{p} \succ \vec{q}$ implies $\vec{q} \in \mathrm{Ach}$, and*
- *$\vec{q} \succ \vec{p}$ implies $\vec{q} \notin \mathrm{Ach}$.*

The Pareto curve is the set

$$\mathrm{Pareto} = \{\vec{p} \in \mathbb{R}^m \mid \vec{p} \text{ is Pareto optimal}\}\,. \tag{64}$$

Lemma 2.5. *It holds:* $\mathrm{Pareto} \subseteq \mathsf{Ach}$.

Specifically, this means that any Pareto-optimal point has a witnessing policy.
Note that the lemma above is not true for arbitrary objectives that go beyond maximal reachability probabilities.

Theorem 2.6. *There is a finite set of points P, such that* Ach *is the smallest convex and downward-closed set containing P.*

Even stronger (and key to proving the theorem above), these finite points are achieved by deterministic and (bounded) finite-memory policies. However, note that $|P|$ can be superpolynomial in the size of the MDP, even if there are just 2 objectives.

2.1.4 Computing Achievable Pointsdiscuss!

We consider two computational problems. The first is a simple decision problem.

> **Problem: Decision Problem - Achievability**
> Given an MDP M with m target sets $T_1, \ldots, T_m$, $T_i \subseteq S$, and a point $\vec{\lambda} \in \mathbb{R}^m$. Decide whether $\vec{\lambda}$ is achievable.

Beyond deciding achievability of individual points, we can aim to compute the Pareto curve (or rather, the achievable points). As the representation of the achievable points can be prohibitively large, we are mostly interested in computing a tight approximation.

> **Problem: Achievable Point Approximation**
> Given an MDP M with m target sets $T_1, \ldots, T_m$, $T_i \subseteq S$. Compute $L, U \subseteq \mathbb{R}^m$ such that:
>
> - $L \subseteq \mathsf{Ach}_M(T_1, \ldots, T_m) \subseteq U$, and
> - $\|U - L\| \leq \varepsilon$, for any given ε and suitable measure on sets.

Computing *the* set of achievable points is a special case where $\varepsilon = 0$. Beyond these queries, one can also try to find particular points on the Pareto curve. This is often called a constrained MDP query.

Preprocessing: Goal unfoldingdiscuss! In this preprocessing step, we reduce reasoning about randomising finite memory policies to reasoning about randomising memoryless policies. The key insight used here is that the memory is not used arbitrarily, but simply to track which set of target states has been visited previously. We then move this memory from the policy into the MDP itself.

Definition 2.4 (Goal unfolding)**.** *Given an MDP $M = \langle S, A, \delta_{\mathsf{orig}} \rangle$ with target states $T_1, \ldots, T_m$. We define* $\mathsf{unfolding}(M)$ *as an MDP*

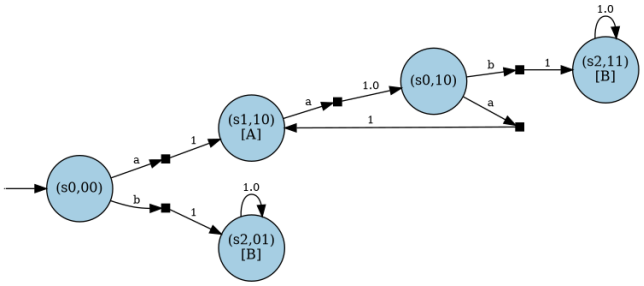
$$\langle S \times \{0,1\}^m, A, \delta_{\mathsf{unf}} \rangle \tag{65}$$

with

$$\delta_{\mathsf{unf}}((s,b),a,(s',c)) = \begin{cases} \delta_{\mathsf{orig}}(s,a,s') & \text{if } c = \mathsf{succ}(b,s') \\ 0 & \text{otherwise.} \end{cases} \tag{66}$$

where $\mathsf{succ}(b,s')_i = b_i \vee [s' \in T_i]$ *for any* i.

Example 2.5. *In the figure below, we apply the unfolding construction on the MDP from Example 2.4.*



The correctness of the construction is summarized by the following theorem:

Theorem 2.7.

- *The set of achievable points in M and* $\mathsf{unfolding}(M)$ *coincide.*
- *Every achievable point for* $\mathsf{unfolding}(M)$ *has a memoryless witness.*

Lemma 2.8. *If all target states are absorbing, then every achievable point has a memoryless witness.*

We remark that the construction is indeed exponential in the number of target sets (only).

Deciding achievability via an LPdiscuss! We assume an unfolded MDP as in Definition 2.4. We furthermore assume the initial state is not in any T_i. We first show that the decision problem, whether a given point is achievable, can be solved with an LP. The (standard) LPs for probabilistic model checking have variables for the probability to reach the target and constraints to encode the action selection. That setup makes it hard to define different reachability probabilities under the same policy. However, we can take a different perspective (in fact, the dual perspective in the linear programming theory) and consider the probability mass flow through the MDP.

Specifically, we define an LP over variables $y_{s,a}$ for every choice (s, a). The idea is that $y_{s,a}$ is to encode the expected number of times that a policy chooses (s, a). Clearly, this number must be non-negative, i.e.,

$$\text{for all } s \in S, a \in A_{\mathsf{en}}(s) : y_{s,a} \geq 0. \tag{67}$$

Remark 2.1 (Choice of variables.)**.** *One can rewrite this to use variables $y_{s,a,s'}$ for every transition $s \xrightarrow{a} s'$, using the equality $y_{s,a,s'} = \delta(s, a, s')y_{s,a}$.*

Under this intuition, we can define auxiliary expressions for the expected number of times a state is left (outflow) and the number of times a state is entered (inflow).

$$f_{out}(s) = \sum_a y_{s,a} + [s = s_{\text{init}}] \tag{68}$$

and

$$f_{in}(s') = \sum_s \sum_a \delta(s, a, s') \cdot y_{s,a}. \tag{69}$$

Intuitively, we want to express that the times a state is entered must be equal to the times we leave the state, i.e.,

$$f_{in}(s) = f_{out}(s). \tag{70}$$

However, the number of times a state is visited under a particular policy may be infinite, if it is part of a maximal end component. We make the following two key observations (from Forejt et al. [2011]):

- if there still is a target state that can be reached, a (Pareto) optimal policy has no incentive to stay in an end-component.
- if there is no target state to be reached, the behavior of a policy (and what states it will reach) is not relevant to our property. We can define the relevant states as follows:

$$S_{\mathsf{rel}} = \{(s, b) \mid \exists c \geq b.(s, c) \text{ reachable from } (s, b).\} \tag{71}$$

[3] We will therefore formulate the following constraint:

[3] This definition is close to, but not the same as $\{s \mid \bigvee_i s_i \in S^{max}_{>0}(\Diamond T_i)\}$: Consider absorbing target states.

$$\text{for all} s \in S_{\mathsf{rel}}, f_{in}(s) = f_{out}(s). \tag{72}$$

The last remaining set of constraints enforces that the policy is actually achievable.

Consider the sets $S_{-i} = \{(s,b) \mid b[i] = 0\}$ and $S_{+i} = \{(s,b) \mid b[i] = 1\}$, i.e., the sets of state where T_i has not and has already been visited.

Observe that along every path, one transitions either zero or one time from S_{-i} to S_{+i}. Thus, the expected number of such transitions coincides with the probability to go from S_{-i} to S_{+i}. As one starts in S_{-i} (the initial state is not part of T_i by assumption), the probability to go from S_{-i} to S_{+i} is the same as the probability to reach S_{+i}, which is the same as the probability to reach a T_i state. This justifies the following constraint:

$$\text{for all } i \leq m \sum_{s \in S_{-i}} \sum_{a} \sum_{s' \in S_{+i}} \delta(s,a,s') \cdot y_{s,a} \geq \lambda_i. \tag{73}$$

If we put these constraints together, we get a sound and complete approach to check achievability.

Theorem 2.9. *Given an Definition 2.4 M and a threshold $\vec{\lambda}$. The LP*

$$\begin{aligned}
&\textit{maximize} \quad 0 \\
&\qquad y_{s,a} \geq 0 && \textit{for all } s \in S, a \in A_{\mathsf{en}}(s), \\
&\qquad f_{in}(s) = f_{out}(s) && \textit{for all } s \in S_{\mathsf{rel}}, \\
&\qquad \sum_{s \in S_{-i}} \sum_{a} \sum_{s' \in S_{+i}} \delta(s,a,s') \cdot y_{s,a} \geq \lambda_i. && \textit{for all } i \leq m
\end{aligned}$$

is feasible iff λ is achievable.

The easiest way to prove the correctness of the approach is by considering this as the dual to the more intuitive standard LP. A witnessing policy π can be extracted by setting

$$\pi(s) =_a y_{s,a}. \tag{74}$$

Example 2.6. *Consider the Example 2.5. The following LP is feasible iff $(0.5, 0.5)$ is achievable.*

```
LP(sense='maximize', objective=0, constraints=[Eq(y_(s0,00)_a + y_(s0,00)_b, 1), Eq(y_(s0,1(
```

As for single-objective MDP model checking, the LP justifies the following statement.

Lemma 2.10. *Deciding achievability can be done in polynomial time.*

We remark that the LP can be extended with an objective such that a solution to the LP yields a Pareto optimal policy.

Computing Pareto curves via weighted reachabilitydiscuss! We now turn our attention to approximating the Pareto curve. We obtain our approximation by computing a subset of the Pareto-optimal policies. Clearly, the convex and downward closure of the points achieved by these policies is an underapproximation. From the fact that the policies we compute are indeed Pareto-optimal, we also obtain a set of unachievable points. The complement of these unachievable points are then an overapproximation of the set of achievable points. As the achievable points can be represented by finitely many vertices, the algorithm we present also terminates for any precision ϵ.

More specifically, the key theorem to the approach is the following:

Theorem 2.11. *Given* $\vec{w} \in [0,1]^m$ *and*

$$\pi \in \arg\max_{\pi'} \sum w_i \mathsf{Pr}_{\pi'}(\Diamond T_i). \tag{75}$$

Then:

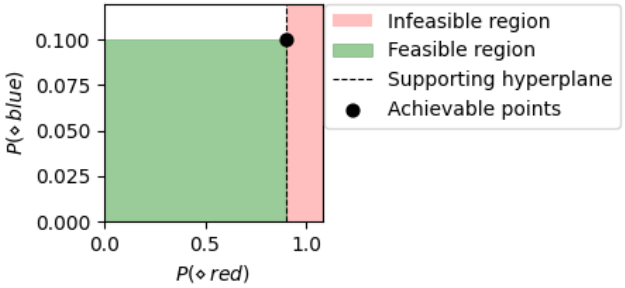
$$\vec{p} = (\mathsf{Pr}_\pi(\Diamond T_1), \dots \mathsf{Pr}_\pi(\Diamond T_m)) \tag{76}$$

is on the boundary of the achievable points. For $w_i \in (0,1]^m$*, the point* $\vec{p}$ *is Pareto-optimal.*

Clearly, $\vec{p}$ is achievable by π. The argument that it is also Pareto optimal (for almost all w) follows from the fact that every point that dominates p yields a higher weighted reachability. Assume that one of those points would be achievable by policy $\hat{\pi}$: Then $\hat{\pi}$ demonstrates that π is not optimal w.r.t. weighted reachability, which is a contradiction to the definition.

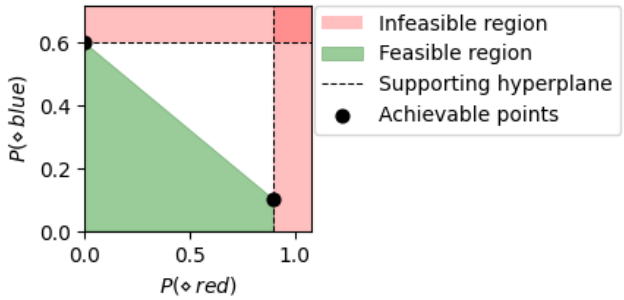
With this theorem, the idea is now to iteratively explore different weight vectors. The theorem above ensures the soundness. Completeness of the algorithm relies on the fact that there are only finitely many policies, but also requires carefully selecting the weight vectors. In particular, while every weight vector different to the previous weight vectors sharpens the overapproximation, we need to be careful to ensure that we find all vertices in finite time. However, if we pick the weight vectors orthogonal to one of the existing faces of the Pareto curve, we only need as many iterations as the number of faces + number of vertices of the exact Pareto curve. The example below also demonstrates this way of picking weight vectors.

Example 2.7. *In the following, we first optimize for the weight vector* $(1,0)$.



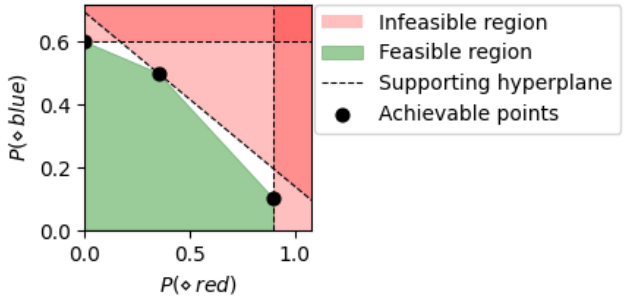
Importantly, we find a policy that achieves $(0.9, 0.1)$ *and update the achievable region accordingly. We can also update the unachievable points (i.e., the complement of the upper bound), by ruling out the hyperplane.*

Next, we optimize for the weight vector $(0, 1)$.

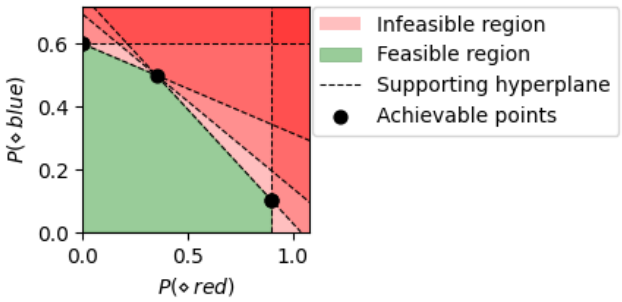


We now see that the convex hull is formed, in particular, points between two achievable points must also be achievable.

Next, we take a weight vector $(0.5, 0.9)$ *orthogonal to the current face between the two Pareto-optimal points:*



This yields one more Pareto optimal point; note that we have now found all vertices of the achievable points — but the algorithm does not know this. In particular, there are two faces (to the left and to the right of the new Pareto point) where there is a gap between under- and overapproximation. By using weight vectors orthogonal to these faces, we can tighten the overapproximation. With these new weight vectors, we find policies whose induced points coincide with previously found points. This also proves that the faces of the underapproximation are Pareto-optimal. That is, there are no further achievable points, as shown in the figure below.



Remark 2.2 (On computing weighted reachability)**.** *The weighted reachability query can be expressed as a total (undiscounted) reward query on the unfolding. Specifically, you get reward w_i for every transition between an S_{-i} and S_{+i}, using the earlier definitions. While the total reward in general is infinite, it is not if the reward collectible in every MEC is zero. As the reward, by construction, is only awarded upon the first entry of some target set, we can only finitely often pick up reward. Therefore, the total reward is finite and computable via all standard approaches, see Quatmann [2023].*

Attention
Currently skipped.

Variant 3: Convex Hull Value Iterationdiscuss!

2.2 Tractable hyperpropertiesdiscuss!

Attention
Work in progress

3 Parametric Markov Decision Processes

In this chapter, we discuss parametric MDPs.

```
import stormpy
import sympy
from fractions import Fraction
import stormvogel as sv
import stormvogel.to_dot
import stormvogel.parametric.region
import stormvogel.teaching as teach
import stormvogel.bird as bird
sympy.init_printing()
from IPython.display import Math
import stormvogel.teaching.bellman as bellman
import stormvogel.examples as examples
```

3.1 What are parametric Markov decision processes?discuss!

Parametric MDPs describe MDPs where the probabilities are not a fixed number, but rather a symbolic expression over some parameters.

Definition 3.1 (Parametric MDPs)**.** *A parametric Markov Decision Process (pMDP) is a tuple*

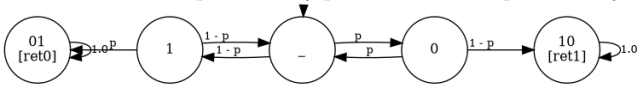
$$\langle S, A, \mathbf{x}, \delta \rangle \tag{77}$$

where S is a finite and nonempty set of states, A is a finite and nonempty set of actions, $\mathbf{x}$ is a finite and nonempty set of parameters, and the transition relation δ is given by $\delta \colon S \times A \nrightarrow S \to \mathbb{Q}(\mathbf{x})$.

Choices are thus not labeled by distributions over states, but by assigning to (successor) states rational functions over some set of parameters. pMDPs can be extended with initial states, labels, target states, etc., just like standard (parameter-free) MDPs.

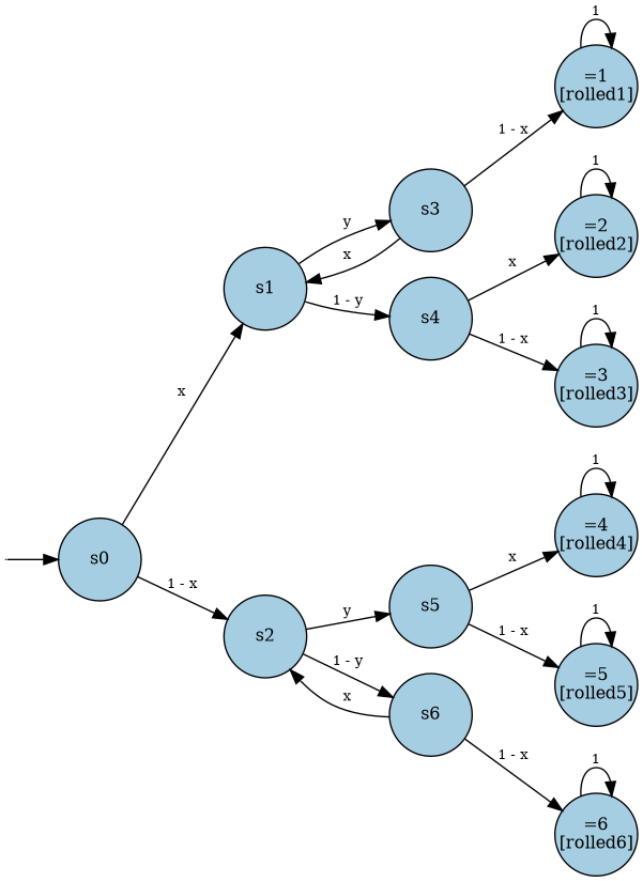
Parametric Markov chains are pMDPs such that in every state, only one action is enabled. We typically denote these as tuples $\langle S, \mathbf{x}, \delta \rangle$.

Example 3.1 (Von Neumann Unbiased Coins)**.** *Say we want to simulate an unbiased, perfectly random coin flip. We only have access to an old, biased coin and can flip it as many times as we want — that is, we have access to an infinite stream of biased random bits, each of which is 0 with some unknown but fixed probability $0 < p < 1$. A simple solution Von Neumann [1951] : Extract the first two bits from the stream; if they are different, return the value of the first; otherwise try again. The following pMC models the protocol. We flip a coin, and obtain a 0 with probability p and a 1 with probability $1 - p$.*



In this chapter, we demonstrate that the probability to reach state 01 is 0.5, irrespectively of the value of p. We note that such an analysis is not possible with the standard s-a-rectangularity assumption in robust models.

Example 3.2 (Parametric Knuth-Yao Dice)**.** *The following example shows the Knuth-Yao die*[4]*, where alternatingly two coins are flipped. The first has a bias x, the second a bias y.*



Analysing the effect of x and y on the probability to reach, say, `rolled3` *is non-trivial. Consider, e.g., the question of how to get this probability above a threshold, i.e., how to maximise the reachability probability. Clearly, x should not be so small that the probability mass at the initial state flows into the right branch. On the other hand, x should also not be too large, as then,* `rolled3` *cannot be reached. The precise optimum depends on the influence of* s_3*, which itself depends on the value of y.*

[4] We note that the precise transition relations differ over the literature: In particular, x and $1 - x$ is sometimes swapped in places.

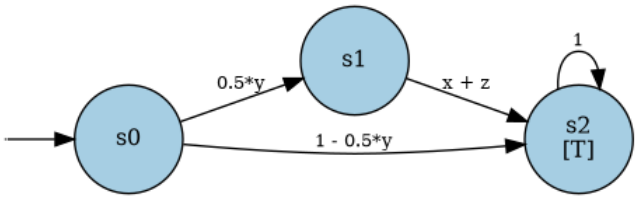
Note
While we discuss the concepts generally for pMDPs, most examples and the algorithms later in this chapter focus on pMCs.

The intention of pMDPs is that by substituting parameters with concrete values, one obtains an MDP. Substitutions of parameters by such values are called well-defined valuations.

Definition 3.2 (Well-defined valuations)**.** *Given a pMDP* $\mathcal{M} = \langle S, A, \mathbf{x}, \delta \rangle$ *over parameters* $\mathbf{x}$*. A valuation* val *is a mapping* $\mathbf{x} \to \mathbb{R}$*. A valuation* val *is well-defined, if*

1. $\delta(s,a)(s')[\mathsf{val}] \geq 0$ *for all* $s, s' \in S$ *and* $a \in A_{\mathsf{en}}(s)$*.*
2. $\sum_{s' \in S} \delta(s,a)(s')[\mathsf{val}] = 1$ *for all* $s \in S$*,* $a \in A_{\mathsf{en}}(s)$*.*

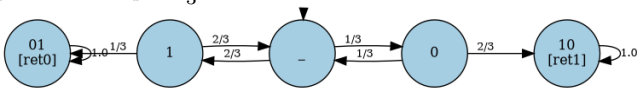
Example 3.3. *For the Example 3.2, the well-defined valuations are all valuations with* $x, y \in [0,1]$*. For the following example, the well-defined valuations must assign* $y \in [0,2]$ *and* $x + z = 1$*.*



Any pMDP together with a well-defined valuation thus describes an MDP.

Definition 3.3 (Induced MDP)**.** *Given a pMDP* $\mathcal{M} = \langle S, A, \mathbf{x}, \delta \rangle$ *and a well-defined valuation* val*, the induced MDP* $\mathcal{M}[\mathsf{val}]$ *is the MDP* $\langle S, A, \delta' \rangle$*, where* $\delta'(s,a)(s') = \delta(s,a)(s')[\mathsf{val}]$ *for all* s, s' *and* $a \in A_{\mathsf{en}}(s)$*.*

Example 3.4 (Induced model)**.** *The following instantiates the pMC from Example* **??** *with* $p = \frac{1}{3}$*.*



An important distinction can be made based on the graph-structure of the pMDP $\mathcal{M}$ and the induced MDP $\mathcal{M}[\mathsf{val}]$. Valuations where these coincide are called *graph-preserving*.

Definition 3.4 (Graph-preserving valuations)**.** *Given a pMDP* $\mathcal{M} = \langle S, A, \mathbf{x}, \delta \rangle$ *over parameters* $\mathbf{x}$*. A valuation* $\mathbf{x} \to \mathbb{R}$ *is graph preserving, if*

$$\delta(s,a)(s')[\mathsf{val}] > 0 \text{ iff } \delta(s,a)(s') \neq 0 \quad \text{for all } s, s' \in S \text{ and } a \in A_{\mathsf{en}}(s)\,. \tag{78}$$

We are often interested in sets of valuations, which are commonly referred to as regions (based on a geometric interpretation).

Definition 3.5 (Well-defined regions)**.** *A set of valuations for a pMDP is called a region. The well-defined region is the set of all well-defined valuations. A region is well-defined, if it is a subset of the well-defined region.*

> **Assumptions**
> We only consider pMDPs with non-empty well-defined regions. Deciding whether this set is empty is hard in general, but it is easy, e.g., if all transitions are affine expressions.

Regions can come in different shapes. Here, we are particularly interested in rectangular regions.

Definition 3.6 (Rectangular regions)**.** *A region R is rectangular, if there exist l_x, u_x for every $x \in \mathbf{x}$, such that*

$$R = \{\mathsf{val} \mid \textit{for all } x \in \mathbf{x} : l_x \leq \mathsf{val}(x) \leq u_x\}. \tag{79}$$

We write $R(x)$ for the interval $[l_x, u_x]$.

Remark 3.1 (Rectangularity)**.** *The notion of rectangularity is not (directly) related to notions like (s,a)-rectangularity for robust MDPs/interval MDPs.*

Any pMDP together with a region of well-defined valuations describes a set of MDPs.

Definition 3.7 (Generated Markov chains.)**.** *Given a pMDP $\mathcal{M}$ and a well-defined region R,*

$$\mathcal{M} \mid R = \{\mathcal{M}[\mathsf{val}] \mid \mathsf{val} \in R\} \tag{80}$$

If R is the well-defined region, we may simply write $\mathcal{M}_{\mathsf{wd}}$.

3.1.1 Types of pMDPsdiscuss!

pMDPs can be distinguished on how parameters occur on transitions. Intuitively, *polynomial pMDPs* have transition probabilities where the denominator is a constant, *affine pMDPs* are polynomial pMDPs where the (total) degree of every transition probability is at most one. Here, we often restrict ourselves to simple pMDPs.

Definition 3.8 (Simple pMDPs)**.** *A pMDP is simple, if for every choice (s, a) and all $s' \in S$:*

$$\delta(s,a)(s') \in \mathbb{Q} \cup \{x, 1-x \mid x \in \mathbf{x}\} \tag{81}$$

and additionally

$$\sum_{s' \in S} \delta(s,a)(s') = 1. \tag{82}$$

As a consequence, non-constant transition probabilities always occur in pairs $x, 1-x$ for $x \in \mathbf{x}$. The models in Example **??** and Example 3.2 are indeed simple pMCs. As we will see below, except for the syntactical form, algorithmically, there is nothing simple about simple pMDPs. A key characteristic of simple pMDPs, and the main reason we use them, is that the set of well-defined regions is simply the unit hypercube and in particular rectangular.

3.1.2 Relation to interval MDPsdiscuss!

Parametric MDPs are closely related to non-rectangular robust MDPs. The syntax of parametric MDPs already naturally indicates a static type of uncertainty. As already highlighted in Example **??**: in contrast to (s,a)-rectangular robust MDPs, parametric MDPs can express that two different transitions must be the same.

The following lemmas between parametric MDPs with rectangular regions and interval MDPs formalizes the relation and can be adapted for various other robust models. Basically, we obtain an iMDP by replacing, for every parameter x, every occurrence of x with $[l_x, u_x]$ and every occurrence of $1-x$ by $[1-u_x, 1-l_x]$.

Definition 3.9 (IMDP abstraction)**.** *Given a simple pMDP $\mathcal{M} = \langle S, A, \delta \rangle$ with a rectangular region R. The interval MDP lifting $\mathcal{M}$ and R is given by*

$$\mathsf{lift}(\mathcal{M}, R) = \langle S, A, \delta' \rangle \tag{83}$$

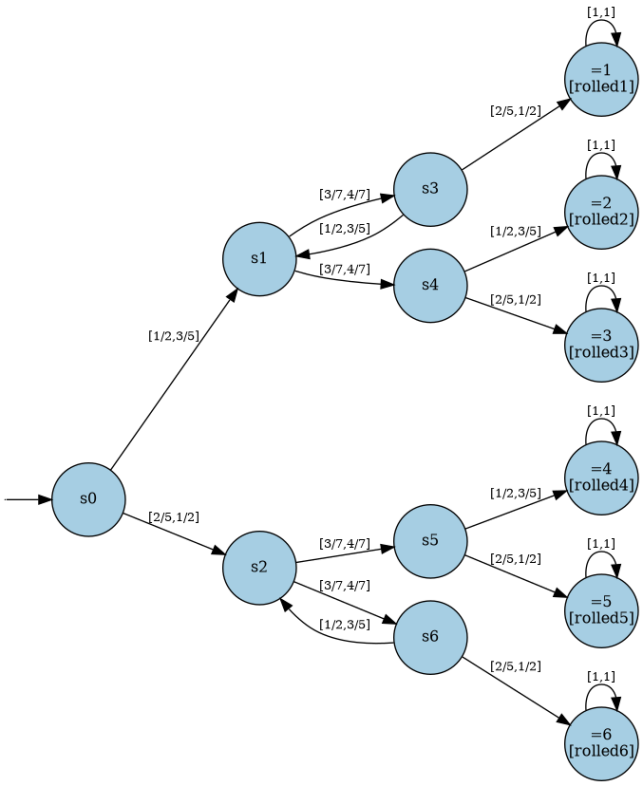
with

$$\delta'(s,a)(s') = \delta(s,a)(s')[x_1 \leftarrow R(x_1), \ldots, x_n \leftarrow R(x_n)] \tag{84}$$

using an interval extension of substitution.

Example 3.5. *We provide an example for Example 3.2 and R with*

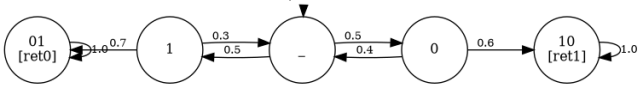
$$\frac{1}{2} \leq R(x) \leq \frac{3}{5}, \frac{3}{7} \leq R(y) \leq \frac{4}{7}. \tag{85}$$



The IMDP is a proper abstraction, when considered via the set of MDPs that can be generated.

Lemma 3.1 (iMDPs abstract pMDPs with rectangular regions)**.** *Given a simplex pMDP $\mathcal{M}$ with a rectangular region R. Then $\mathcal{M} \mid R \subseteq \mathsf{lift}(\mathcal{M}, R)$.*

Example 3.6 (iMDPs abstract pMDPs)**.** *Consider $\mathcal{M}$ as in Example* **??** *and R with $0.4 \leq R(p) \leq 0.7$. Let $\mathcal{M}'$ be the lifting of that pMC and R. The following MC is in $\mathcal{M}'$ but not in $\mathcal{M} \mid R$.*



Note that vice versa, we can express every interval MDP as a parametric MDP.

Definition 3.10 (pMDP representation for iMDPs)**.** *Given an interval MDP $\mathcal{M} = \langle S, A, \delta \rangle$, we define the underlying variables*

$$\mathbf{x} = \{x_{(s,a,s')} \mid s, s' \in S, a \in A\}. \tag{86}$$

For $\mathcal{M}$, we then define

- *the matching pMDP* $\mathsf{parametric}(\mathcal{M}) = \langle S, A, \mathbf{x}, \delta' \rangle$*, with*

$$\delta(s,a)(s') = \begin{cases} x_{s,a,s'} & \text{if } \delta(s,a)(s') > 0 \\ 0 & \text{otherwise.} \end{cases} \tag{87}$$

- *and the matching region* R *with*

$$R(x_{(s,a,s')}) = \delta(s,a)(s'). \tag{88}$$

Lemma 3.2 (Matching pMDPs generate the same MDPs as iMDPs)**.** *Let* $\mathcal{M}$ *be an iMDP and* $\mathcal{M}'$ *the matching pMDP and* R *the matching region. Then:*

$$\mathcal{M} = \mathcal{M}' \mid R. \tag{89}$$

3.1.3 Relation to policy-induced modelsdiscuss!

Given an MDP M[5], we can represent an Definition 1.7 of a memoryless, randomising policy via M_{MR}. This Markov chain is obtained by picking weights for every state-action combination, $w_{s,a}$.

Definition 3.11 (Corresponding pMC for MDP)**.** *Given an MDP* $M = \langle S, A, \delta \rangle$*, we define the underlying variables*

$$\mathbf{x} = \{ w_{(s,a)} \mid s \in S, a \in A_{\mathsf{en}}(s) \}. \tag{90}$$

For M*, we then define the corresponding pMC* $\mathsf{corresponding}(M) = \langle S, A, \mathbf{x}, \delta' \rangle$*, with*

$$\delta(s,a)(s') = \begin{cases} \sum_a w_{s,a} \cdot \delta(s,a)(s') & \text{if } a \in A_{\mathsf{en}}(s) \\ 0 & \text{otherwise.} \end{cases} \tag{91}$$

From the definitions, the following follows immediately:

Lemma 3.3. *For any MDP:*

$$M_{\mathsf{MR}} = \mathsf{corresponding}(M)_{\mathsf{wd}}. \tag{92}$$

The value of this construction is twofold:

- We can mildly modify it to add additional constraints on the policies, which then translate to constraints on the parameter region of interest.
- We can rephrase problems on pMDPs as problems on pMCs.

[5]The construction can be easily lifted to parametric MDPs. We omit that for conciseness.

3.2 Solution functions and their computationdiscuss!

The value of an MDP is a useful abstraction to refer to a quantity of interest in parameter-free models. In parametric models, the value is no longer a constant but rather a function over the parameters.

Specifically, for any fixed pMDP, every valuation maps to an MDP. For any fixed objective (such as maximal reachability objectives), an MDP maps to a value. Solution functions combine this by fixing the pMDP and the objective. That is, the solution function (for a pMDP and an objective) maps every valuation to a value, i.e., solution functions are expressions over the parameters. By definition, substituting the parameters in the solution function yields the same result as substituting the parameters in the pMDP and applying model checking. In what follows, we only study solution functions for maximal reachability probabilities.

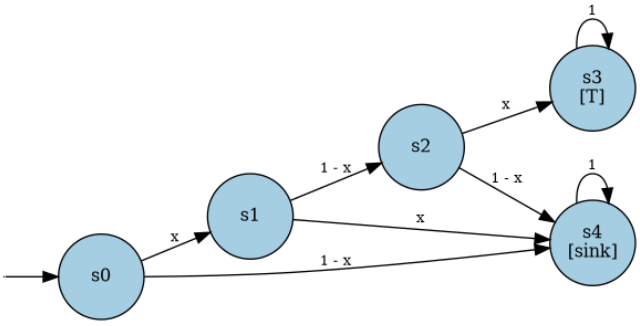
Definition 3.12 (Solution function (MaxReachProb))**.** *Given a pMDP $\mathcal{M}$ with initial states and targets T, and a well-defined region R. The solution function* $\mathsf{sol}\colon R \to [0,1]$ *is defined as* $\mathsf{sol}(\mathsf{val}) = \mathsf{Pr}^{\max}_{\mathcal{M}[\mathsf{val}]}(\Diamond T)$.

Example 3.7. *Consider the following pMC.*

```
pmc = sv.model.new_dtmc(create_initial_state=False)
x = pmc.declare_parameter("x")

s0 = pmc.new_state(["init"], friendly_name="s0")
s1 = pmc.new_state(friendly_name="s1")
s2 = pmc.new_state(friendly_name="s2")
s3 = pmc.new_state(["T"], friendly_name="s3")
s4 = pmc.new_state(["sink"], friendly_name="s4")

pmc.set_choices(s0, [(x,     s1), (1 - x, s4)])
pmc.set_choices(s1, [(1 - x, s2), (x,     s4)])
pmc.set_choices(s2, [(x,     s3), (1 - x, s4)])
pmc.set_choices(s3, [(1, s3)])
pmc.set_choices(s4, [(1, s4)])
stormvogel.to_dot.plot_model_pydot(pmc)
```



*The solution function is -x**3 + x**2, which is obtained by multiplying the transition probabilities along the single path from initial state to target.*

Remark 3.2 (Uniqueness of the solution functions)**.** *The literature is inconsistent wrt the definition of a solution function: There is the solution function for a given region R, which is only defined on R, or a solution function that coincides with the solution function on R and is arbitrarily defined otherwise. In particular, algorithms to compute solution functions compute functions that are defined on $\mathbb{R}^m$, but which are only solution functions on the well-defined region (or a subset thereof).*

3.2.1 Shape and size of solution functionsdiscuss!

It is sometimes interesting to reduce the domain of the solution function, e.g., to discuss the shape of the function, e.g., to talk about the representation of solution functions on graph-preserving regions. The shape and size of solution functions is not arbitrary.

Solution functions on parametric Markov chains.discuss! We state two key facts about solution functions for pMCs.

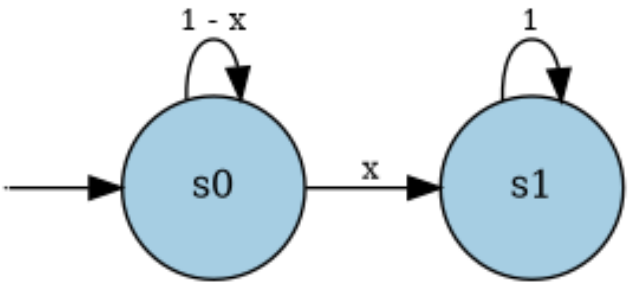
Theorem 3.4 (Shape of solution functions)**.** *Given a pMC. The solution function for $\mathcal{M}$ on well-defined region R is guaranteed to be:*

- *lower semicontinuous and*
- *a rational function, if R is graph-preserving and*
- *a polynomial, if $\mathcal{M}$ is polynomial and acyclic.*

While there are direct proofs for the latter two points, we will justify them as a consequence of the correctness of the state elimination algorithm. The first point follows from

1. the continuity of the rational function (notice that the denominator will never be zero on a well-defined point)
2. the fact that removing transitions can only reduce the reachability probability.

Example 3.8. *This example demonstrates that the solution function on the well-defined region is in general not continuous.*



In particular, the probability to leave the initial state and thus reach the target is one, if $x \in (0, 1]$. If $x = 0$, the probability is zero.

Theorem 3.5 (Size of solution functions)**.** *For acyclic simple pMCs with n states and k parameters, the size of the solution function (written as sum of terms) is polynomial in n and exponential in k, in the worst case.*

In particular, this result also means that every algorithm to compute solution functions is necessarily exponential in the number of parameters.

Solution functions on parametric Markov decision processes.discuss! The solution function for a pMDP is then the finite maximum over these rational functions.

3.2.2 Computing solution functions for pMCsdiscuss!

We present the state-elimination algorithm for computing solution functions. Note that this algorithm is not optimal in terms of complexity, in particular, it can run in exponential time for a fixed number of parameters. Nevertheless, it is arguably the simplest algorithm. The algorithm is closely connected to the state elimination algorithm for nondeterministic finite automata.

State elimination computes solution functions with a series of transformations on the parametric Markov chain.

1. **Make the target state absorbing**. This simplifies the description below.
2. **Self-loop elimination**.
   Given any state s which is not a sink-state. We can eliminate self-loops on state s by rescaling all other outgoing transitions.
3. **Transition shortcutting**. Take a state s' without self-loop. Replace a transition from s to s' by new or updated transitions from s to all successors of s'. The probability from s to $\hat{s}$, with $\hat{s}$ a successor of s, is updated to

$$\delta(s, \hat{s}) \leftarrow \delta(s, \hat{s}) + \delta(s, s') \cdot \delta(s', \hat{s}). \tag{93}$$

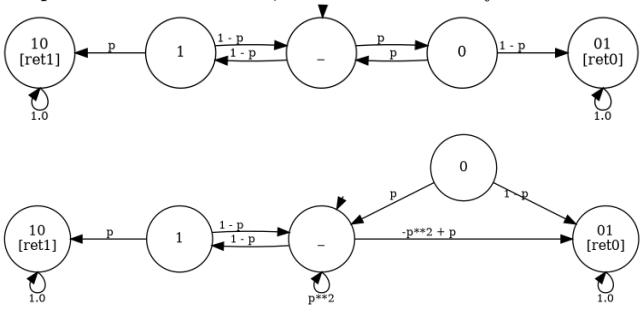
   The transition probability $\delta(s, s')$ is updated to 0.

4. **State removal** (optional). Take any state with no incoming transition. Remove the state and all outgoing transitions.

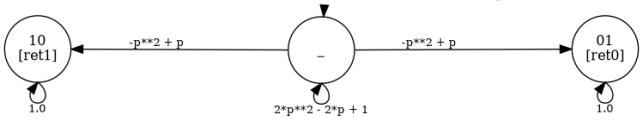
None of the steps above affects the reachability probability in any Markov chain generated by the pMC. In particular, self-loop elimination is correct: taking a loop with transition probability < 1 infinitely often has probability zero.

Eliminating states can be applied to any non-initial, non-sink state s. It runs transition shortcutting on all incoming transitions of s and then optionally removes a state s. The state elimination algorithm removes self-loops whenever possible and then eliminates non-initial, non-sink states.

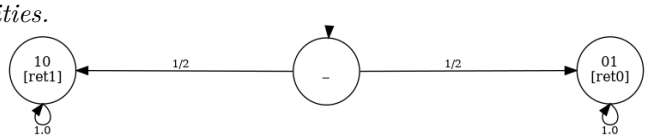
Example 3.9. *We apply elimination steps on Example* **??***, repeated here for convenience. The targets are already absorbing and state s_0 has no self-loop, so we can eliminate the transition from the initial state to s_0. This introduced a self-loop at the initial state, and a transition from initial state to the state 01.*



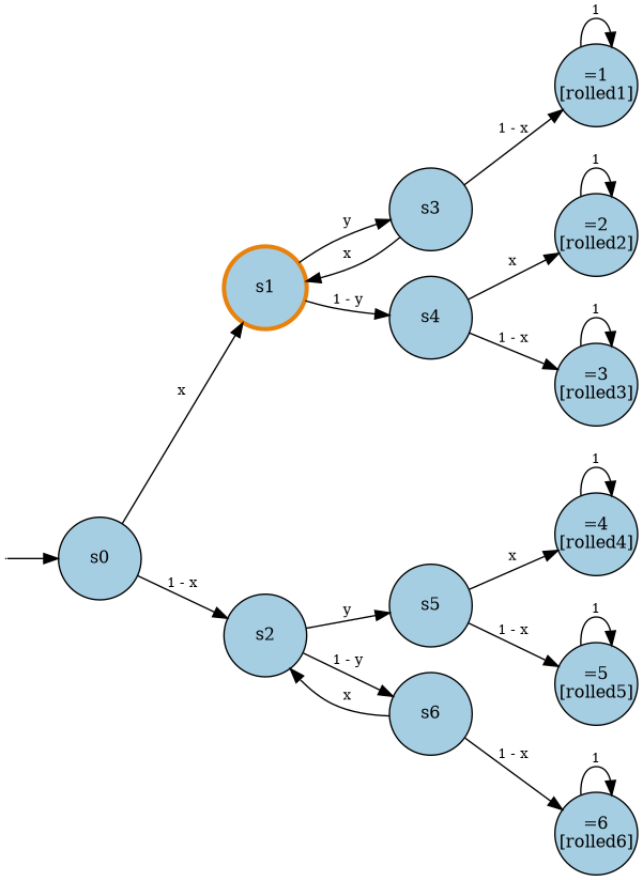
Now, as there are no more incoming states in s_0, we can eliminate the state and then also do the same elimination for s_1 (which also has no self-loop).

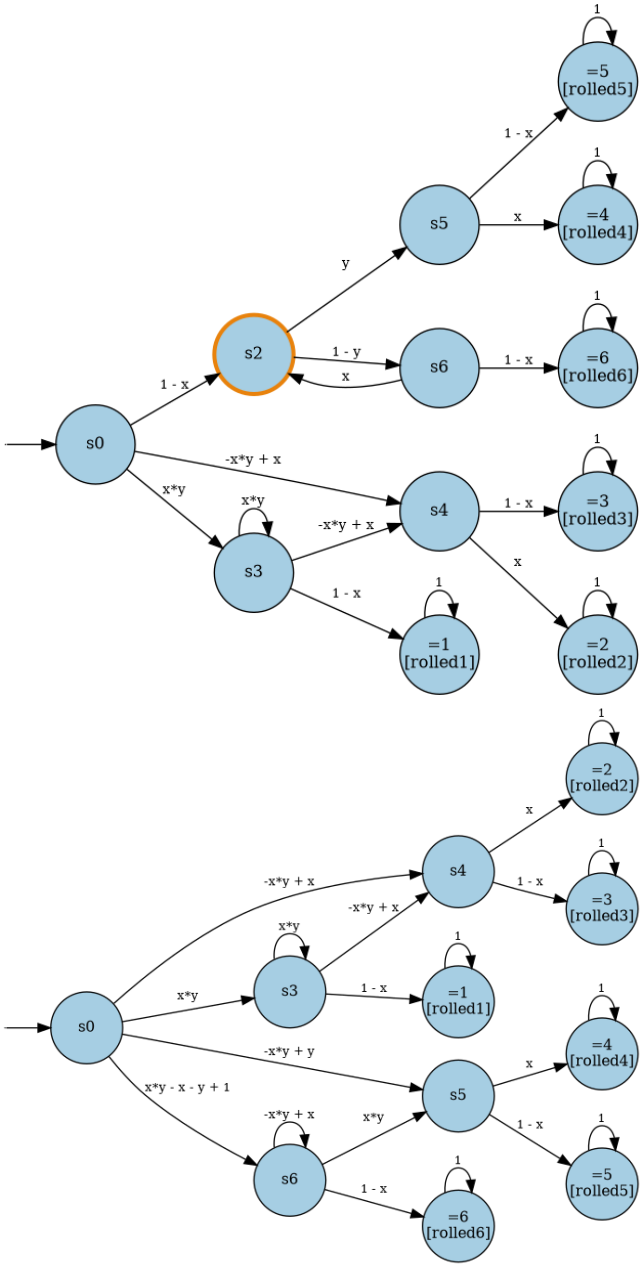


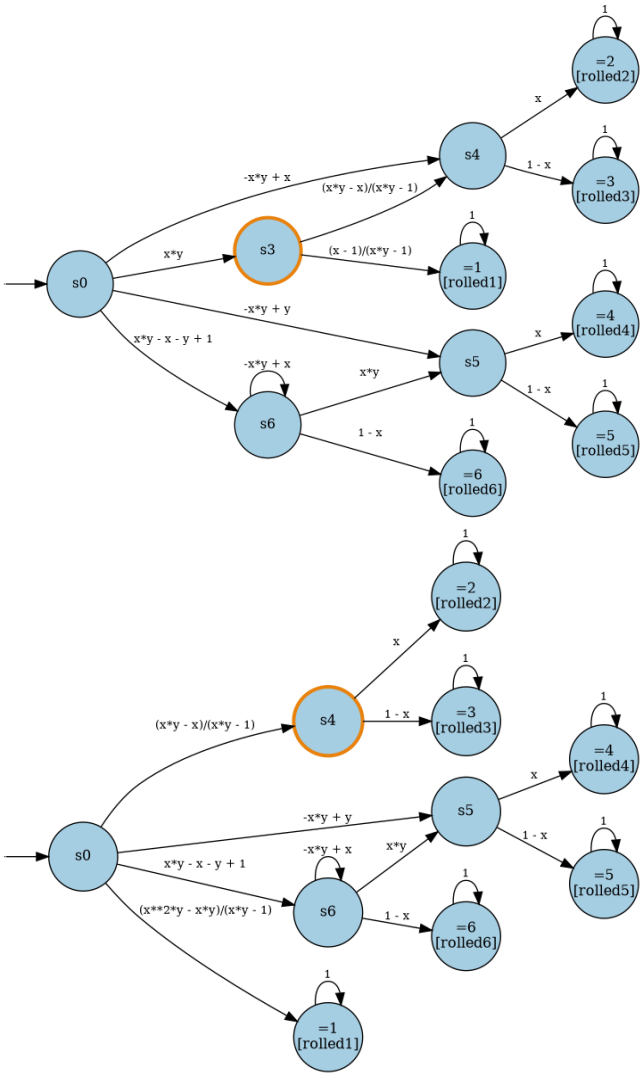
Finally, we remove the self-loop from the initial state to rescale the probabilities.

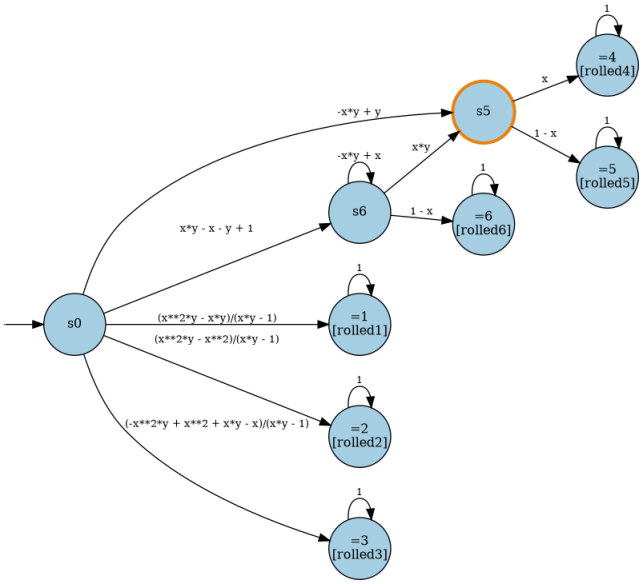


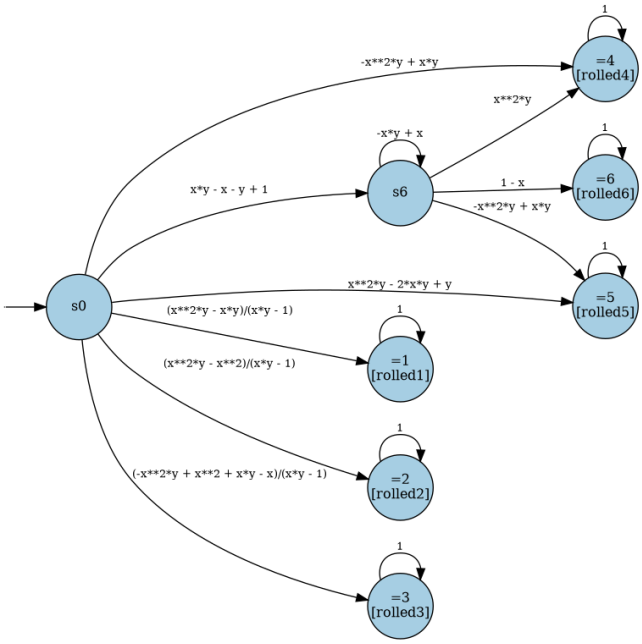
Example 3.10. *We now also run state elimination for Example 3.2.*

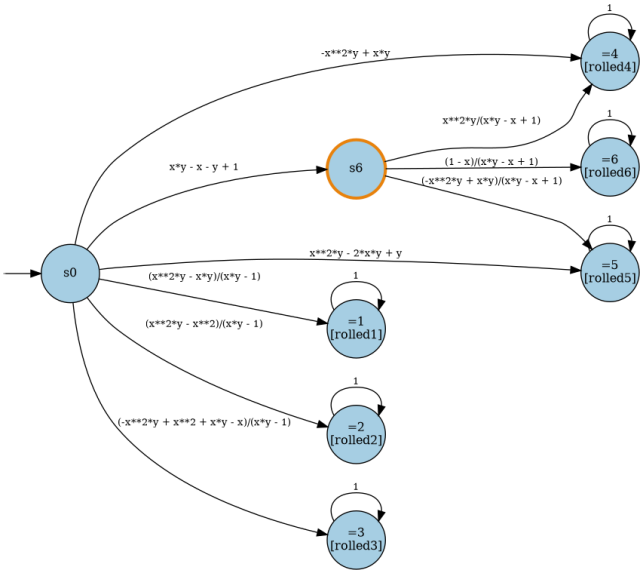

s0
s1
s2
s3
s4
s5
s6
x
1 - x
y
x
1 - y
x
1 - x
y
1 - y
x
x
1 - x
1 - x
1
=1
[rolled1]
=2
[rolled2]
=3
[rolled3]
=4
[rolled4]
=5
[rolled5]
=6
[rolled6]

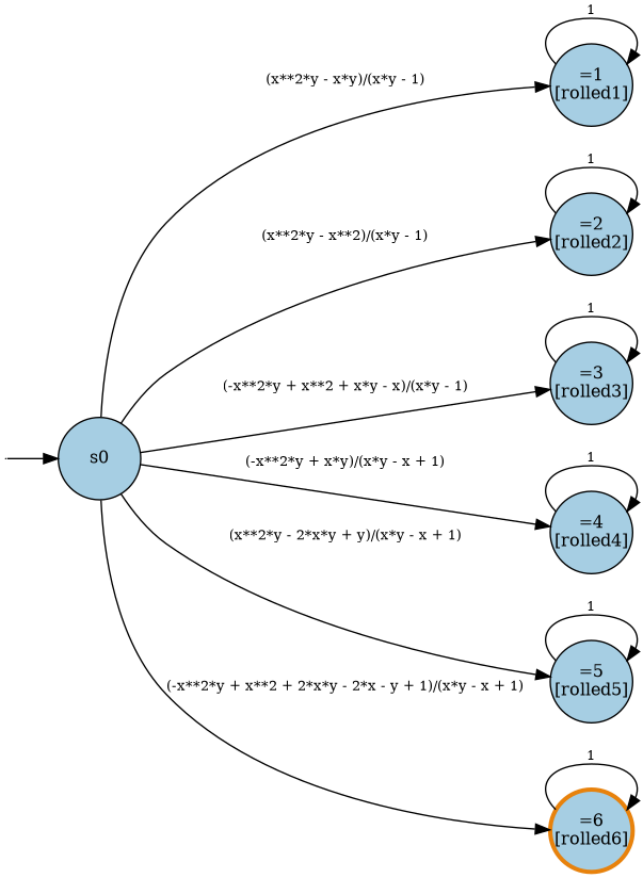

s0
s2
s5
s6
s4
s3
=5
[rolled5]
=4
[rolled4]
=6
[rolled6]
=3
[rolled3]
=1
[rolled1]
=2
[rolled2]
1 - x
x
y
1 - y
-x*y + x
x*y
1


s0
s3
s4
s5
s6
=1
[rolled1]
=2
[rolled2]
=3
[rolled3]
=4
[rolled4]
=5
[rolled5]
=6
[rolled6]
x*y
-x*y + x
(x*y - x)/(x*y - 1)
(x - 1)/(x*y - 1)
-x*y + y
x*y - x - y + 1
x
1 - x
1
(x**2*y - x*y)/(x*y - 1)


s0
s5
s6
=1
[rolled1]
=2
[rolled2]
=3
[rolled3]
=4
[rolled4]
=5
[rolled5]
=6
[rolled6]
-x*y + y
x*y - x - y + 1
(x**2*y - x*y)/(x*y - 1)
(x**2*y - x**2)/(x*y - 1)
(-x**2*y + x**2 + x*y - x)/(x*y - 1)
-x*y + x
x*y
1 - x
x
1


s0
s6
=1
[rolled1]
=2
[rolled2]
=3
[rolled3]
=4
[rolled4]
=5
[rolled5]
=6
[rolled6]
-x**2*y + x*y
x**2*y - 2*x*y + y
x*y - x - y + 1
(x**2*y - x*y)/(x*y - 1)
(x**2*y - x**2)/(x*y - 1)
(-x**2*y + x**2 + x*y - x)/(x*y - 1)
-x*y + x
x**2*y
1 - x
-x**2*y + x*y
1


s0
s6
=1
[rolled1]
=2
[rolled2]
=3
[rolled3]
=4
[rolled4]
=5
[rolled5]
=6
[rolled6]
-x**2*y + x*y
x*y - x - y + 1
x**2*y/(x*y - x + 1)
(1 - x)/(x*y - x + 1)
(-x**2*y + x*y)/(x*y - x + 1)
x**2*y - 2*x*y + y
(x**2*y - x*y)/(x*y - 1)
(x**2*y - x**2)/(x*y - 1)
(-x**2*y + x**2 + x*y - x)/(x*y - 1)
1


s0
=1
[rolled1]
=2
[rolled2]
=3
[rolled3]
=4
[rolled4]
=5
[rolled5]
=6
[rolled6]
(x**2*y - x*y)/(x*y - 1)
(x**2*y - x**2)/(x*y - 1)
(-x**2*y + x**2 + x*y - x)/(x*y - 1)
(-x**2*y + x*y)/(x*y - x + 1)
(x**2*y - 2*x*y + y)/(x*y - x + 1)
(-x**2*y + x**2 + 2*x*y - 2*x - y + 1)/(x*y - x + 1)
1

3.2.3 Computing solution functions for pMDPsdiscuss!

Solution functions for MDPs are harder to compute due to the maximum over the policies. In principle, they can be computed by enumerating over all memoryless, deterministic policies and for each induced MC computing the solution function. Some alternatives have been proposed in the literature that encode a pMDP into a pMC by replacing action choices with parameters. The details of such approaches are beyond the scope of these notes.

3.3 Parameter Synthesis Questionsdiscuss!

While solution functions may be seen as the natural generalisation to values, computing them is often prohibitively expensive. This motivates the study of problems and algorithms that do not take the solution function as intermediate solution. We give an overview and first focus on the most common setup in a probabilistic model checking context, before mentioning further problems. All problem statements here refer to reachability probabilities, but they can be lifted to expected rewards or other temporal properties.

3.3.1 Feasible parameters and verified parametric modelsdiscuss!

The key questions in parametric models are the existence and absence of parameters such that the induced models satisfy a property.

Feasibility problemsdiscuss! Feasibility problems ask for the existence of parameters. Classical examples for feasibility problems are the existence of hyperparameters in, e.g., randomized algorithms and distributed protocols. Like for standard MDPs, we can distinguish between the intent of the policy. For parametric Markov chains, these problems coincide — we simply talk about the feasibility problem.

Problem: Angelic feasibility
Given a pMDP $\mathcal{M}$ with targets T, a well-defined region R, a *threshold* $\lambda \in \mathbb{Q}$, and $\bowtie \in \{\leq, \geq\}$, decide whether

$$\exists \mathcal{M} \in \mathcal{M} \mid R . \exists \pi \in \Pi . \mathsf{Pr}^{\pi}_{\mathcal{M}}(\Diamond T) \bowtie \lambda \tag{94}$$

Problem: Demonic feasibility
Given a pMDP $\mathcal{M}$ with targets T, a well-defined region R, a *threshold* $\lambda \in \mathbb{Q}$, and $\bowtie \in \{\leq, \geq\}$, decide whether

$$\exists \mathcal{M} \in \mathcal{M} \mid R . \forall \pi \in \Pi . \mathsf{Pr}^{\pi}_{\mathcal{M}}(\Diamond T) \bowtie \lambda \tag{95}$$

Specifically, in angelic feasibility, we assume that the policy is helping to achieve the property. A few remarks are in order:

- Angelic feasibility can be reformulated into a pMC problem via this construction.
- Natural variations to the problems above are the optimization variants for all these problems.
- It suffices to consider memoryless deterministic policies.

Verification problemsdiscuss! We can consider the duals of the feasibility problems which intend to verify that for every MDP, a property holds. Verification problems are often studied to ensure robustness against variations in the probability distributions.

Problem: Angelic verification
Given a pMDP $\mathcal{M}$ with targets T, a well-defined region R, a *threshold* $\lambda \in \mathbb{Q}$, and $\bowtie \in \{\leq, \geq\}$, decide whether

$$\forall \mathcal{M} \in \mathcal{M} \mid R. \exists \pi \in \Pi. \mathsf{Pr}^{\pi}_{\mathcal{M}}(\Diamond T) \bowtie \lambda. \tag{96}$$

Problem: Demonic verification
Given a pMDP $\mathcal{M}$ with targets T, a well-defined region R, a *threshold* $\lambda \in \mathbb{Q}$, and $\bowtie \in \{\leq, \geq\}$, decide whether

$$\forall \mathcal{M} \in \mathcal{M} \mid R. \forall \pi \in \Pi. \mathsf{Pr}^{\pi}_{\mathcal{M}}(\Diamond T) \bowtie \lambda. \tag{97}$$

Some remarks again:

- The dual to angelic (demonic) feasibility is demonic (angelic) verification, respectively.
- Demonic verification can be formulated over pMCs with a lot of additional parameters, just like angelic feasibility above.
- It suffices to consider memoryless deterministic policies.

Partitioningdiscuss! In the problem statements above, the region of valuations is fixed. A classical problem in the analysis of parametric MDPs is to find the set of valuations that induce an MDP that satisfies the property.

Definition 3.13 (Safe region)**.** *Given a pMDP $\mathcal{M}$ with targets T, a well-defined region R, a threshold $\lambda \in \mathbb{Q}$, and $\bowtie \in \{\leq, \geq\}$, the safe region is*

$$R_{\mathsf{safe}} = \{\mathsf{val} \in R \mid \Pr^{\pi}_{\mathcal{M}}[\mathsf{val}](\Diamond T) \bowtie \lambda\}, \tag{98}$$

and the unsafe region is $R_{\mathsf{unsafe}} = R \setminus R_{\mathsf{safe}}$

To check whether any individual region is safe (or unsafe), i.e., whether $R = R_{\mathsf{safe}}$, is a standard parametric verification problem, see above.

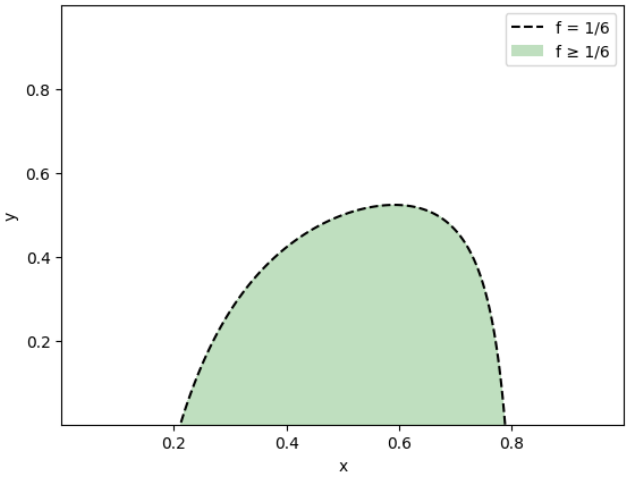
Problem: Exact partitioning
Given a pMDP $\mathcal{M}$ with targets T, a well-defined region R, a *threshold* $\lambda \in \mathbb{Q}$, and $\bowtie \in \{\leq, \geq\}$, compute R_{safe}.

However, the exact representation of this set is often infeasible. In fact, the following lemma clarifies the shape of an exact solution to the partitioning problem:

Lemma 3.6 (Characterisation of safe region)**.** *Given a pMDP $\mathcal{M}$ with targets T, a well-defined region R, a threshold $\lambda \in \mathbb{Q}$, and $\bowtie \in \{\leq, \geq\}$ Let* sol *be the solution function on R for reaching T. Then:*

$$R_{\mathsf{safe}} = \{\mathsf{val} \mid \mathsf{sol}[val] \bowtie \lambda\}. \tag{99}$$

Example 3.11. *The following example continues with Example 3.2 and gives the safe region for the reachability probability to rolled3 over Fraction(1, 6).*



*In fact, the boundary is given by the solution function: -1/6 + (-x**2*y + x**2 + x*y - x)/(x*y - 1).*

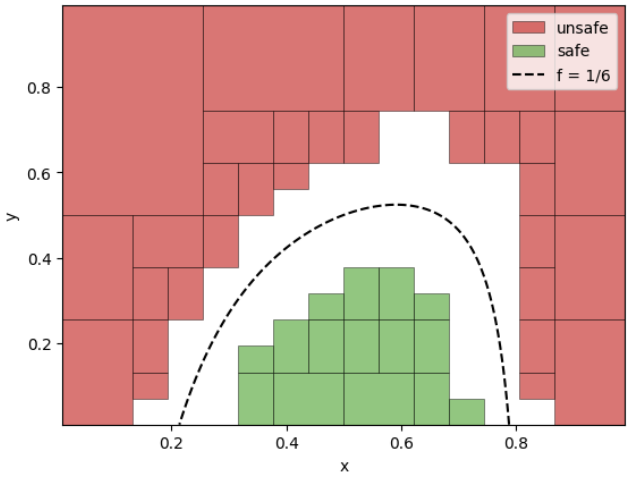
In particular, given the continuity of the solution function on the well-defined region, the expression $\mathsf{sol} - \lambda = 0$ describes exactly the boundary between the safe and unsafe region! To avoid the limitations due to the size of the solution function, a popular approach is to approximate this set.

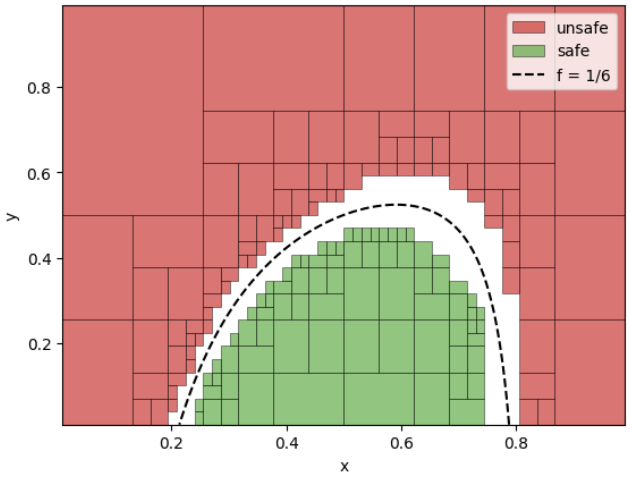
Problem: Approximate partitioning
Given a pMDP $\mathcal{M}$, a well-defined region R, a *threshold* $\lambda \in \mathbb{Q}$, and $\bowtie \in \{\leq, \geq\}$, compute sequences $R_{\mathsf{safe},i}$ and $R_{\mathsf{unsafe},i}$ such that:

- $R_{\mathsf{safe},i} \subset R_{\mathsf{safe}}$ for all i,
- $R_{\mathsf{unsafe},i} \subset R_{\mathsf{unsafe}}$ for all i,
- $\lim_{i\to\infty} R_{\mathsf{safe},i} = R_{\mathsf{safe}}$,
- $\lim_{i\to\infty} R_{\mathsf{unsafe},i} = R_{\mathsf{unsafe}}$.

The notions of limits here are well-defined on mild assumptions on the shapes of the individual regions.

Example 3.12. *We now approximate the safe region from Example 3.11. The following figures shows a coarse and a finer approximation. We also plot the boundary of the safe region for convenience.* R_{safe} *and* R_{unsafe} *are both represented by a union of finitely many rectangular regions.*





As the example already indicates, the common approach to compute such approximations is by splitting the region R into smaller regions and using a verification algorithm on these smaller regions.

3.3.2 Beyond standard questionsdiscuss!

The literature considers other variations, e.g., whether the reachability probability is monotonically increasing in a parameter.

Robust policiesdiscuss! Above, we have always quantified first over the parameters / the generated MDPs and only then over the policies. That is, the policies can be different in different MDPs. For many planning problems, one wants to find one policy that can reach a target in every environment.

Problem: Robust policies
Given a pMDP $\mathcal{M}$, a well-defined region R, a *threshold* $\lambda \in \mathbb{Q}$, and $\bowtie \in \{\leq, \geq\}$, decide whether

$$\exists \pi \in \Pi . \exists \mathcal{M} \in \mathcal{M} \mid R.\mathsf{Pr}^{\pi}_{\mathcal{M}}(\Diamond T) \bowtie \lambda. \tag{100}$$

In interval MDPs, computing robust policies is equally hard as the verification problem. However, in parametric MDPs, finding robust policies is much harder - the policies typically require infinite amounts of memory and even when restricting oneself to the memoryless policies with a single parameter is NP-hard. Robust policies are currently not in the scope of these notes.

Distributions over parametersdiscuss! Furthermore, while in these notes, we assume existential or universal quantification over the parameters, research

also assumes a distribution (a prior) over parameter values and can ask whether a randomly generated MDP (according to the prior distribution) will satisfy a specification.

3.4 Algorithms for feasibility and verificationdiscuss!

We have already discussed how exact partitioning relates to the solution function and how approximate partitioning is typically realised by applying verification algorithms on subregions. We now focus on algorithms to solve feasibility and verification problems on pMCs. Many ideas carry over to pMDPs, but they are not discussed here. Furthermore, feasibility and verification are duals of each other, thus a complete approach for feasibility is also a complete approach for verification. However, positive answers on feasibility are significantly easier to obtain, and leads to different algorithmic choices.

3.4.1 Encoding into the existential theory of the realsdiscuss!

Lifting the equation systemdiscuss! The first idea is to lift the linear equation system (with variables for the reachability probability from a state and constant transition probabilities as coefficients) to a nonlinear equation system where the parameters are multiplied by the variables for the states.

As a precondition to lifting these encodings, recall that we must precompute the zero states. We use the following lemma that allows us to compute the zero states for all graph-preserving valuations by picking the induced pMC for any fixed graph-preserving valuation.

Lemma 3.7. *Given a pMC $\mathcal{M}$ with graph-preserving region R. For two valuations $\mathsf{val}_1, \mathsf{val}_2 \in R$, it holds that:*

$$\{s \mid \mathsf{Pr}^s_{\mathcal{M}[\mathsf{val}_1]}(\Diamond T) = 0\} = \{s \mid \mathsf{Pr}^s_{\mathcal{M}[\mathsf{val}_2]}(\Diamond T) = 0\}. \tag{101}$$

We can therefore uniquely define $S^{gp}_{=0}$ as the set of zero states for any graph-preserving valuation.

This allows us to state the correctness of a lifted encoding.

Theorem 3.8. *Given a pMC $\mathcal{M}$ with states $s_0, \dots s_n$, parameters $\vec{x}$, initial state s_{init} and targets T, a graph-preserving region R, a threshold $\lambda \in \mathbb{Q}$. The*

statement

$$\exists \vec{x} \exists p_{s_0} \dots p_{s_n} \tag{102}$$

$$\bigwedge_{x \in \vec{x}} x \in R(x) \qquad \wedge \tag{103}$$

$$\bigwedge_{s \in S^{gp}_{=0}} p_s = 0 \qquad \wedge \tag{104}$$

$$\bigwedge_{s \in T} p_s = 1 \qquad \wedge \tag{105}$$

$$\bigwedge_{s \in S \setminus (T \cup S^{gp}_{=0})} p_s = \sum_{s'} \delta(s, s') \cdot p_{s'} \qquad \wedge \tag{106}$$

$$p_{s_{init}} \geq \lambda \tag{107}$$

is valid iff there is a feasible parameter instantiation.

The proof simply observes that for any assignment to the parameters, this correctly provides the standard linear equation system.

Example 3.13. `FeasibilityProblem(parameter_bounds={x: (1/100, 99/100), y: (1/100, 99/100)},`

The statement above is a statement in the existential theory of the reals (ETR), which allows addition and multiplication of real-valued variables, along with inequalities and equality thereof, and any Boolean combinations of such (in)equalities. The validity of statements in the existential theory of the reals is decidable in the complexity class $\exists\mathbb{R}$, which is contained in PSPACE. The class is also NP-hard. Practically, these problems can be solved by computer algebra systems, or by SMT-solver of the quantifier-free fragment of nonlinear arithmetic (like z3 and cvc5). However, the practical performance of such approaches is limited to few variables. As the variables in these encodings are the sum of parameters and states, this in particular also limits the amount of states that can efficiently be supported.

Remark 3.3. *The theorem and the encoding can be generalized to any well-defined valuation by encoding the set of zero states into the statement.*

Using the solution functiondiscuss! Therefore, it is attractive to not use a variable for every state. In fact, we can simply use the solution function and an ETR solver as for the Lemma 3.6:

Lemma 3.9. *Given a pMC $\mathcal{M}$ with targets T, a well-defined region R, a threshold $\lambda \in \mathbb{Q}$, and $\bowtie \in \{\leq, \geq\}$ Let* sol *be the solution function on R for reaching T. Then: The statement*

$$\exists x \in X \mathsf{sol} \geq \lambda \tag{108}$$

is valid iff there is a feasible parameter instantiation.

Practically, the downsides of this approach are twofold: First, one needs to compute the solution function first, which is often quite expensive. Second, while the number of variables is now equal to the number of parameters, all structure within the problem is lost to the solver.

Hardnessdiscuss! The limited performance of solvers for ETR makes it natural to ask whether we can maybe hope to do better. However, the feasibility problem is ETR-complete, i.e., ETR solvers are in some sense adequate tools, just like LP solvers are adequate for MDPs.

3.4.2 Good guessingdiscuss!

To answer *yes* to the feasibility problem (or to find a feasible valuation), one simply needs to guess a good solution. Then, one can verify that this solution is indeed valid efficiently using standard model checking. Practically relevant feasibility solvers therefore rely on such guesses. Different ideas have been used, often based on linearizations of the nonlinear equation system or on gradient descent.

3.4.3 Parameter liftingdiscuss!

Let us turn our attention to a complete approach for feasibility/verification. Rather than relying on the ETR encodings, we will use an abstraction-refinement procedure akin to branch-and-bound methods.

Assumptions
For exposition, we assume simplex pMDPs with MaxReachProps and rectangular graph-preserving regions.

For that, we are interested in the value range of a region:

$$V_{\mathcal{M},R} = [\min_{M \in \mathcal{M}|R} \mathsf{Pr}^{\max}_M(\Diamond T), \max_{M \in \mathcal{M}|R} \mathsf{Pr}^{\max}_M(\Diamond T)]. \tag{109}$$

Computing this value range for parametric models is intuitively complicated because due to parameter dependencies, we cannot reason locally, as already pointed out in Example 3.2. On the other hand, in absence of such dependencies, reasoning becomes much simpler. Thus, instead of the pMDP $\mathcal{M}$, we analyze $\mathsf{lift}(\mathcal{M}, R)$ The value range of this iMDP is

$$V_{\mathcal{M}} = [\min_{M \in \mathsf{lift}(\mathcal{M},R)} \mathsf{Pr}^{\max}_M(\Diamond T), \max_{M \in \mathsf{lift}(\mathcal{M},R)} \mathsf{Pr}^{\max}_M(\Diamond T)]. \tag{110}$$

Due to Lemma 3.1, $V_{\mathcal{M},R} \subseteq V_{\mathsf{lift}(\mathcal{M},R)}$.

Now, to solve the verification problem, we must prove that $V_{\mathcal{M}}(R) \geq \lambda$. If we compare $V_{\mathsf{lift}(\mathcal{M},R)}$ with λ, there are 3 cases:

1. $V_{\mathsf{lift}(\mathcal{M},R)} \geq \lambda$, then also $V_{\mathcal{M},R} \geq \lambda$ and we have proven the property.

2. $V_{\mathsf{lift}(\mathcal{M},R)} \leq \lambda$, then also $V_{\mathcal{M},R} \leq \lambda$ and we have refuted the property.

3. $\lambda \in V_{\mathsf{lift}(\mathcal{M},R)}$. In this case, we cannot conclude anything.

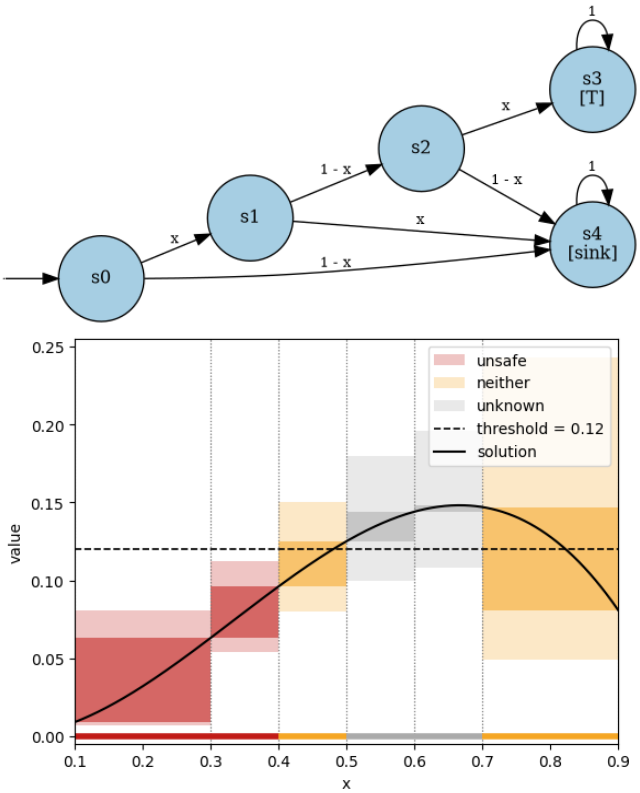
To handle the third case, we split the region, as justified by the following observation:

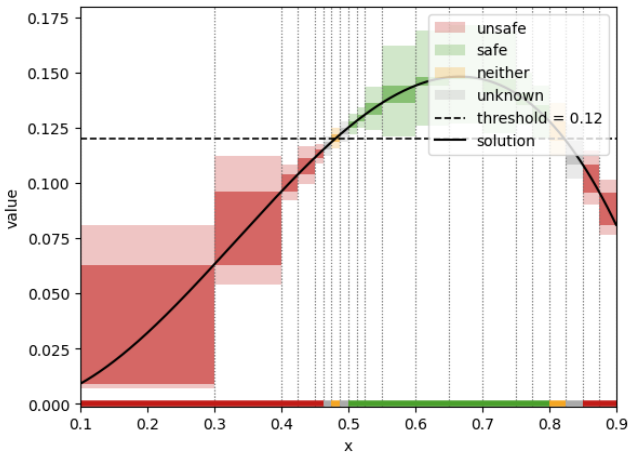
Lemma 3.10. *Let $R_1, R_2 \subseteq R$ be regions with $R_1 \cup R_2 = R$.*

$$V_{\mathcal{M},R} \geq \lambda \quad \textit{iff} \quad V_{\mathcal{M},R_1} \geq \lambda \textit{ and } V_{\mathcal{M},R_2} \geq \lambda. \tag{111}$$

Under mild assumptions, the interval MDP abstraction is less coarse for smaller regions. In particular, due to the smoothness of the solution function on the graph-preserving region, if we converge towards point regions, we also converge against point intervals for both the value of a region of a PMDP and the value of its abstraction.

Example 3.14. *We reconsider the following example on region $R = [0.1, 0.9]$. We show the situation after a few iterations and after more iterations. On the x-axis, you see the different regions. The range of the lifted iMC is visualised, the color indicates which of the three cases above applies for this value range, in comparison with the threshold.*



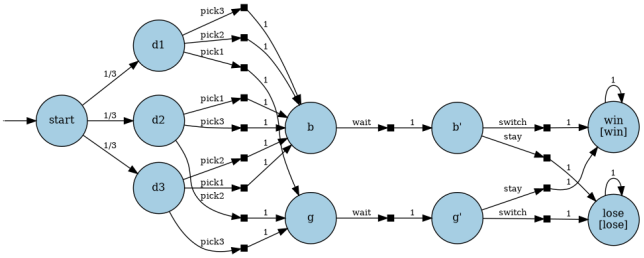

unsafe
safe
neither
unknown
threshold = 0.12
solution
value
x
0.175
0.150
0.125
0.100
0.075
0.050
0.025
0.000
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
0.9

4 Partially observable MDPs

In this chapter, we discuss POMDPs.

4.1 What are POMDPs?discuss!

Example 4.1. *Consider the Monty-hall problem: You play in a game show, there are three doors, behind one of these doors is a car, behind the other doors are goats. The assumption here is that you want to win the car. You must pick a door, and without any information here, any pick is equally good. Now the game master interfers: They reveal that after a randomly picked door different then the one you picked is a goat. The game master now asks you whether you want to stick to your choice or change. The simple question is? Should you change. To answer this question, you may be tempted to construct the following MDP:*



Let's look at the optimal probability to win (and the policy that is optimal):

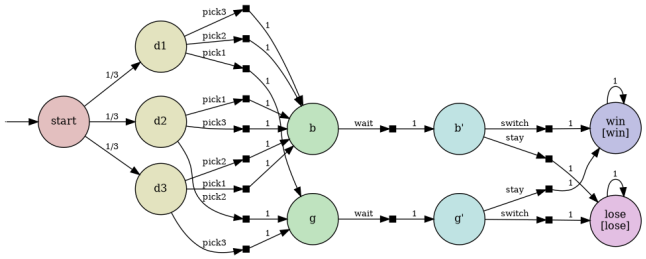
```
Optimal win probability: 1.0
```

The fully observable MDP admits a policy that wins with probability 1: stay at g' (the initial pick was correct) and switch at b' (the initial pick was wrong). This policy, however, relies on distinguishing states g' and b' — that is, knowing whether the initial door choice was correct. In the actual game, the player has no access to this information.

The problem in the example above is that the agent (the player in the game show) can make decisions based on unavailable information. This analysis is thus inadequate for any purposes where you want to prove that a sensor-based system can achieve a goal or that an attacker cannot steal sensitive information. To overcome these limitations, POMDPs have been introduced. They model explicitly that at every state, only limited information is available.

Example 4.2. *In the POMDP, observations hide the car's location. States d_1, d_2, d_3 share observation `pick?`: the player knows they must pick a door but cannot see where the car is. States g and b share observation `switch?`: after picking, the player cannot tell whether they chose correctly. States g' and b' share observation `show!`: after the host reveals a goat, the player still cannot tell whether their original pick was right.*

Observation-based policies can therefore only act on these three observation phases. At observation `pick?` *the player chooses one of three doors; at observation* `show!` *they choose to stay or switch. This gives* $3 \times 2 = 6$ *deterministic observation-based policies (and their randomised convex combinations).*



Definition 4.1 (Partially Observable MDP)**.** *A partially observable MDP (POMDP) is a triple* $\langle M, Z, \mathsf{obs} \rangle$*, where* M *is an MDP with states* S*,* Z *is a finite nonempty set of observations, and* $\mathsf{obs}\colon S \to \mathsf{Distr}(Z)$ *is a state-based observation function.*

Remark 4.1. *Beyond state-based observations, the literature also uses transition-based observations where information is shared when taking a transition. We do not use such models here.*

> **Assumption**
> From here on, assume deterministic observations with the shape $\mathsf{obs}\colon S \to Z$. Any stochastic observation function can be transformed into a determinsitic one using a polynomial blowup.

Given the assumption on deterministic observations, we write

$$S_z = \{s \in S \mid \mathsf{obs}(s) = z\} \tag{112}$$

> **Assumption**
> For all $s, s' \in S_z$, we assume $A_{\mathsf{en}}(s) = A_{\mathsf{en}}(s')$. We then write $A_{\mathsf{en}}(z)$ for the common set of enabled actions at observation z.

Definition 4.2 (Observation trace)**.** *Given any path*

$$\xi = s_0 \xrightarrow{a_0} s_1 \xrightarrow{a_1} s_2 \xrightarrow{a_2} \dots \tag{113}$$

the observation trace is

$$\mathsf{obs}(\xi) = \mathsf{obs}(s_0) \xrightarrow{a_0} \mathsf{obs}(s_1) \xrightarrow{a_1} \mathsf{obs}(s_2) \xrightarrow{a_2} \dots \tag{114}$$

Definition 4.3 (Observation-based policies)**.** *A policy* π *is observation-based, if for* $\xi, \xi' \in \mathsf{Paths}$

$$\mathsf{obs}(\xi) = \mathsf{obs}(\xi') \text{ implies } \pi(\xi) = \pi(\xi'). \tag{115}$$

4.1.1 Reachability probabilities in POMDPsdiscuss!

Assumptions
We assume that target states have a unique and exclusive target observation z_{target}, i.e., for $s \in T$, $\mathsf{obs}(s) = z_{\text{target}}$ and for $s \notin T$, $\mathsf{obs}(s) \neq z_{\text{target}}$.

Problem: Standard verification problem for reachability probabilities
Given POMDP M, a set of target states T, a *threshold* $\lambda \in \mathbb{Q}$ and $\bowtie \in \{\leq, \geq\}$, decide whether

$$\mathsf{Pr}^{\pi}_{M}(\Diamond T) \bowtie \lambda \tag{116}$$

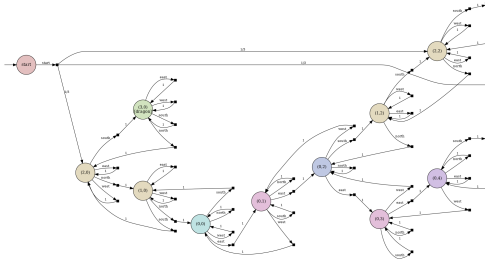
holds for **all** observation-based policies.

Problem: Standard planning problem for reachability probabilities
Given POMDP M, a set of target states T, $\lambda \in \mathbb{Q}$ and $\bowtie$, compute **some** observation-based policy π such that

$$\mathsf{Pr}^{\pi}_{M}(\Diamond T) \bowtie \lambda. \tag{117}$$

Problem statementsdiscuss!

Difficulty of the problemdiscuss! Before we go into details, let us review the famous cheese-maze POMDP. Even in this simple POMDP, adding memory is important to act optimally.



Example 4.3 (Optimal observation-based policies require memory)**.**
Consider the cheese maze POMDP with three vertical corridors. The agent starts uniformly in one of the (upper) cells of the three corridors; not on the top row. All cells within the corridors share the same observation NS, so from the current observation alone the agent cannot tell which corridor it is in.

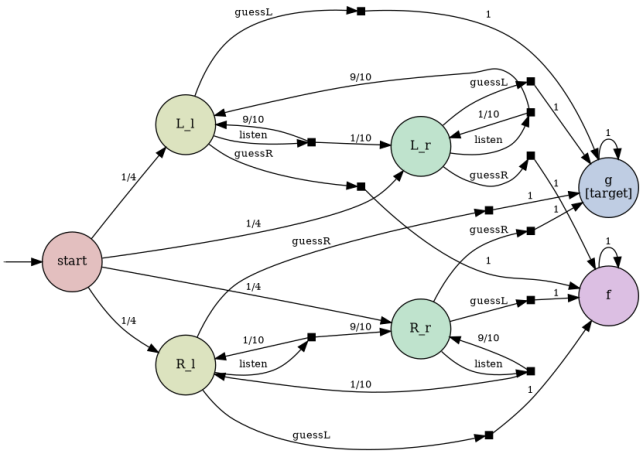
The middle corridor leads to the cheese; the outer two lead to the dragon. It is possible to reach the cheese with probability one. An optimal strategy first moves upwards. Once it hits the top row, it has uniquely determined the agent's position. It then moves to the correct corridor and descends. A memoryless policy must apply the same action in every corridor — with observation NS. Then, the agent must never descend while there is positive probability of being in a dragon corridor. But a policy that never descends never reaches the cheese either. Thus, a memoryless policy cannot reach the cheese with probability one.

Sometimes, memoryless policies are sufficient, but randomization may then still be necessary:

Example 4.4 (Optimal memoryless policies require randomization)**.** *Consider the cheese maze, and assume the agent starts in one of the two top-corner cells (observations ES or SW, which are distinguishable). Even though the starting position is known, reaching the middle corridor requires traversing EW cells on the top row, which are all indistinguishable. In a memoryless setting, the action taken in an EW cell must be the same no matter how the agent arrived there. A deterministic memoryless policy fixes a single direction—left or right—at every EW cell, so from one of the two starting corners the agent always moves away from the middle corridor and can never reach the cheese. A randomising policy, by contrast, can assign a small probability to descending at each NS cell and a high probability to continuing along the top row. This way, whenever the agent happens to be above the cheese corridor, there is a positive probability of descending correctly, so the cheese is reached with probability one eventually. No deterministic memoryless policy achieves this.*

In general, we cannot give an upper bound on the amount of memory necessary.

Example 4.5 (Supremum not attained)**.** *Consider a POMDP with states L_l, L_r, R_l, R_r, goal state g, and failure state f. States L_l and R_l share observation l; states L_r and R_r share observation r: the agent sees only the last signal, not whether the hidden type is L or R. The start state distributes uniformly over all four, so both hidden types are initially equally likely to be true. The three actions are* listen*,* guessL*, and* guessR*. Under* listen*, L-states transition to L_l with probability $\frac{9}{10}$ and L_r with probability $\frac{1}{10}$; R-states transition to R_r with probability $\frac{9}{10}$ and R_l with probability $\frac{1}{10}$ — so the signal is informative but noisy. Under* guessL*, the agent moves to g from any L-state and to f from any R-state;* guessR *is symmetric.*



The supremum equals 1. *For any n, the agent can apply* listen *n times and then guess the majority signal seen. By the law of large numbers, the probability of guessing correctly approaches 1 as $n \to \infty$, so $\sup_\pi \mathsf{Pr}^\pi(\Diamond g) = 1$.*

No policy attains it. *Every finite observation history has positive probability under both hidden states: for any sequence $o_1 \dots o_k$,*

$$\mathsf{Pr}(o_1 \dots o_k \mid L) > 0 \quad \textit{and} \quad \mathsf{Pr}(o_1 \dots o_k \mid R) > 0. \tag{118}$$

Therefore, whenever a policy guesses, there is positive probability of being in the wrong hidden state and reaching f. Thus $\mathsf{Pr}^\pi(\Diamond g) < 1$ for every policy π, and the supremum is not attained.

This example is key to proving the following general theorem.

Theorem 4.1 (Optimal policies do not need to exist)**.** *For a POMDP M and target T, the supremum*

$$\sup_{\pi \ \textit{observation-based}} \mathsf{Pr}^\pi_M(\Diamond T) \tag{119}$$

need not be attained: there may be no observation-based policy π^* achieving this value.

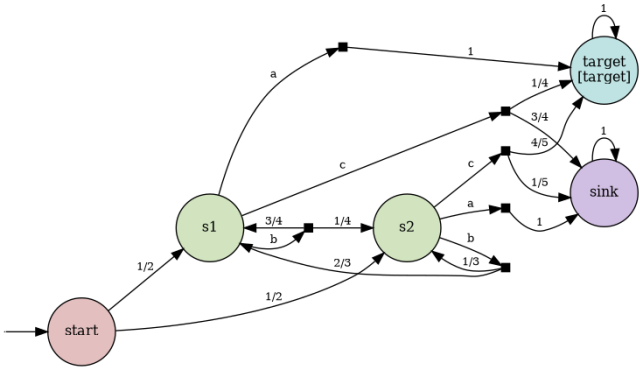
Indeed, the unbounded amount of memory is at the root of an undecidability statement.

Theorem 4.2 (Undecidable)**.** *The standard verification and planning problems for POMDPs with reachability objectives are undecidable.*

4.1.2 Beliefsdiscuss!

Above, we have seen that optimal policies must rely on information from the history. However, it turns out that the precise history is not necessary for optimal decisions. Instead, the policy depends on the belief, i.e., a distribution that captures the likelihood of being in a particular state, given the history of observations seen and actions taken.

Example 4.6 (4-State POMDP — Running Example)**.** *We use the following POMDP as a running example throughout this chapter. States s_1 and s_2 share observation z and are therefore indistinguishable to the agent. Three actions are available: a leads deterministically to the target from s_1 and to the sink from s_2; b mixes the hidden state; c gives a partial chance of reaching the target from either state.*



Belief updatesdiscuss!

Definition 4.4 (Belief update)**.** *In a belief b, taking action a yields an updated belief as follows. Let*

$$P(b, a, z') := \sum_{s \in S_{\mathsf{obs}(b)}} b(s) \cdot \sum_{s' \in S_{z'}} \delta(s, a, s') \tag{120}$$

denote the probability of observing $z' \in Z$ upon taking action $a \in A$ in belief b.

If $P(b, a, z') > 0$, the corresponding successor belief $b' = b \mid a, z'$ with $\mathsf{obs}(b') = z'$ is defined component-wise as

$$b \mid a, z'(s') := \frac{\sum_{s \in S_{\mathsf{obs}(b)}} b(s) \cdot \delta(s, a, s')}{P(b, a, z')} \tag{121}$$

for all $s' \in S_{z'}$. Otherwise, $b \mid a, z'$ is undefined.

Definition 4.5 (Belief for a path)**.** *Given a $\xi = s_0 \xrightarrow{a_0} s_1 \xrightarrow{a_1} \dots$, the belief $b_\xi \in \mathsf{Distr}(S)$ is defined inductively:*

- *For a path of length zero $\xi = s_0$: $b_\xi(s) = [s = s_0]$.*
- *For an extended path:*

$$b_{\xi \cdot a \cdot s'} = b_\xi \mid a, \mathsf{obs}(s') \tag{122}$$

.

Naturally, the belief for two paths with the same observation trace coincides.

Example 4.7 (Belief updates on the 4-state POMDP)**.** *We compute belief updates on the running example (Example 4.6), starting from the uniform belief* $b_{\text{init}} = \{s_1 \mapsto \frac{1}{2}, s_2 \mapsto \frac{1}{2}\}$.

Step	*Action*	*Observation*	*Belief*
0	—	—	*$\textstyle\left\{\, s1 \mapsto \tfrac{1}{2}, \quad s2 \mapsto \tfrac{1}{2} \,\right\}$*
1	*b*	*z*	*$\textstyle\left\{\, s1 \mapsto \tfrac{17}{24}, \quad s2 \mapsto \tfrac{7}{24} \,\right\}$*
2	*b*	*z*	*$\textstyle\left\{\, s1 \mapsto \tfrac{209}{288}, \quad s2 \mapsto \tfrac{79}{288} \,\right\}$*

In the supremum-not-attained POMDP (Example 4.5), repeated applications of listen *concentrate the belief towards the true hidden state.*

Step	*Action*	*Observation*	*Belief*
0	—	—	*$\textstyle\left\{\, L_l \mapsto \tfrac{1}{2}, \quad R_l \mapsto \tfrac{1}{2} \,\right\}$*
1	*listen*	*l*	*$\textstyle\left\{\, L_l \mapsto \tfrac{9}{10}, \quad R_l \mapsto \tfrac{1}{10} \,\right\}$*
2	*listen*	*l*	*$\textstyle\left\{\, L_l \mapsto \tfrac{81}{82}, \quad R_l \mapsto \tfrac{1}{82} \,\right\}$*
3	*listen*	*l*	*$\textstyle\left\{\, L_l \mapsto \tfrac{729}{730}, \quad R_l \mapsto \tfrac{1}{730} \,\right\}$*
4	*listen*	*l*	*$\textstyle\left\{\, L_l \mapsto \tfrac{6561}{6562}, \quad R_l \mapsto \tfrac{1}{6562} \,\right\}$*

The belief weight on the L variants grows with each l observation, but never reaches 1 — reflecting that the hidden state can never be determined with certainty.

Belief-based policiesdiscuss! Given this notion of belief and the mapping from an observation trace to a belief, we now introduce policies that make decisions solely based on the belief of a path/of a trace.

Definition 4.6 (Belief-based policies)**.** *A policy π is belief-based if there exists $\hat{\pi} \colon \mathsf{Distr}(S) \to \mathsf{Distr}(A)$ such that*

$$\pi(\xi) = \hat{\pi}(b_\xi) \quad \textit{for all } \xi \in \mathsf{Paths}. \tag{123}$$

Lemma 4.3 (Belief-based implies observation-based)**.** *Every belief-based policy is observation-based.*

By induction on the definition of b_ξ, equal observation traces yield equal beliefs: $\mathsf{obs}(\xi) = \mathsf{obs}(\xi')$ implies $b_\xi = b_{\xi'}$. Thus any belief-based policy is observation-based.

The converse does not hold in general: an observation-based policy may use the full observation history in a way that distinguishes two paths with equal beliefs, e.g., by counting steps or recalling past actions.

Theorem 4.4 (Belief-based policies are sufficient (for reachability))**.** *For any POMDP M,*

$$\sup_{\pi \text{ observation-based}} \mathsf{Pr}^\pi_M(\Diamond T) = \sup_{\pi \text{ belief-based}} \mathsf{Pr}^\pi_M(\Diamond T). \tag{124}$$

The belief MDP discuss! Since belief-based policies are sufficient and the belief captures everything about the history that is relevant for future decisions, we can replace hidden states entirely with belief distributions, reducing the POMDP to a standard (infinite-state) MDP.

Definition 4.7 (Belief MDP)**.** *The belief MDP of POMDP $\mathcal{M}$ is the (infinite-state) MDP*

$$\mathcal{M}^B = (\mathcal{B}, A, \delta^B), \tag{125}$$

with transition function

$$\delta^B(b, a, b') = [\, b' = b \mid a, z' \,] \cdot P(b, a, z') \tag{126}$$

where $z' = \mathsf{obs}(b')$.

For a given initial state s_{init}, the initial belief is

$$b_{\text{init}} = \{s_{init} \mapsto 1\} \tag{127}$$

This belief MDP is naively continuous state, but for finite POMDPs, the reachable state space of a belief MDP is infinite but countable.

Lemma 4.5 (Belief MDP size and shape)**.**

- *The (reachable fragment) of a belief MDP has a countable number of states.*
- *If the POMDP is acyclic, the belief MDP is acyclic and finite.*

The optimal reachability probability in a belief MDP coincides with the optimal reachability probability in the POMDP.

Theorem 4.6. *For POMDP M with target $T \subseteq S$, let $T^B = \{b \in \mathcal{B} \mid \text{supp}(b) \subseteq T\}$. Then*

$$\sup_{\pi \text{ observation-based}} \mathsf{Pr}^\pi_M(\Diamond T) = \sup_{\pi \in \Pi_{md}} \mathsf{Pr}^\pi_{\mathcal{M}^B}(\Diamond T^B). \tag{128}$$

In particular, it suffices to consider deterministic belief-based policies!

Belief-value functionsdiscuss! For the belief MDP, we can consider the standard value function that maps a state of that MDP to a value of interest, in this case the maximal reachability probability. This function thus maps POMDP beliefs to probabilities, $V : \mathcal{B} \to [0, 1]$. This function is of interest independent of the belief MDP.

Two structural properties are particularly relevant. First, under any fixed observation-based policy the belief-value function is affine in the belief.

Lemma 4.7. *For any observation-based policy π, $V^\pi(b) = \alpha^\pi \cdot b$ for some vector $\alpha^\pi \in [0, 1]^S$.*

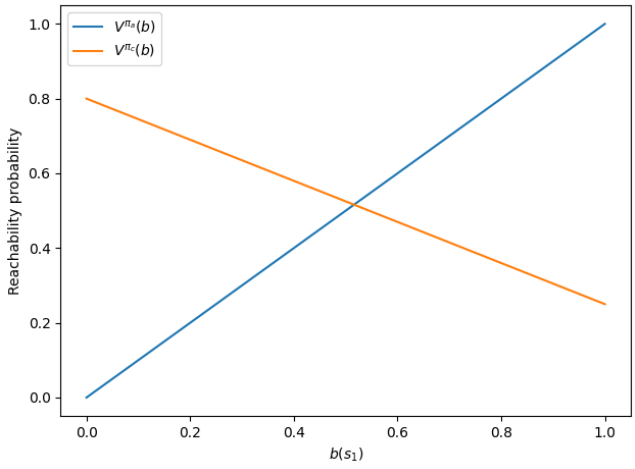
For a fixed observation-based policy π, the action taken at each step is determined by the observation-action history alone. The probability of any state trajectory $\xi = s_0 \xrightarrow{a_0} s_1 \xrightarrow{a_1} \cdots$ under π is therefore $b(s_0) \cdot \prod_t \delta(s_t, a_t, s_{t+1})$, which is linear in $b(s_0)$. Summing over all target-reaching trajectories gives

$$V^\pi(b) = \sum_{s \in S} b(s) \cdot \underbrace{\mathsf{Pr}^\pi(\Diamond T \mid s_0 = s)}_{\alpha^\pi(s)} = \alpha^\pi \cdot b. \tag{129}$$

Example 4.8 (Alpha vectors on the 4-state POMDP)**.** *Consider the two single-step memoryless policies on Example 4.6: always take action a, or always take action c. For policy π_a: from s_1 action a reaches the target with probability 1; from s_2 with probability 0. For policy π_c: from s_1 action c reaches the target with probability $\frac{1}{4}$; from s_2 with probability $\frac{4}{5}$. The two alpha vectors are therefore $\alpha^{\pi_a} = (1, 0)$ and $\alpha^{\pi_c} = (\frac{1}{4}, \frac{4}{5})$, and the value functions*

$$V^{\pi_a}(b) = b(s_1), \qquad V^{\pi_c}(b) = \tfrac{1}{4}\, b(s_1) + \tfrac{4}{5}\, b(s_2) \tag{130}$$

are both linear in b.



Such an affine function is called an *alpha vector*:

Definition 4.8 (Alpha vector)**.** *An alpha vector is a function* $\alpha\colon S \to [0,1]$*. For a belief* $b \in \mathcal{B}$*, the value under* α *is the dot product, i.e.,*

$$\alpha \cdot b = \sum_{s \in S} \alpha(s) \cdot b(s). \tag{131}$$

Lemma 4.8. *The belief-value function for a fixed observation-based policy* π *is an alpha-vector.*

Second, the optimal belief-value function is the pointwise supremum over alpha vectors:

$$V^*(b) = \sup_{\alpha \in \Gamma} \alpha \cdot b, \tag{132}$$

where Γ is the set of alpha vectors representing all relevant (in general, belief-based deterministic) policies.

Lemma 4.9. $V^*(b)$ *is convex.*

Note that if it suffices to only consider a finite set of such policies, $V^*(b)$ is piecewise linear.

4.2 Algorithms for POMDPsdiscuss!

Since exact computation is undecidable (Theorem 4.2) and an optimal policy need not exist (Theorem 4.1), in practice one computes bounds on the optimal value $V^*(b_{\text{init}})$. We construct a number of different algorithms, for either computing lower bounds (mostly for planning) or upper bounds (mostly for verification) of the maximal reachability probability. The ideas can be lifted to minimal reachability probabilities and to expected reachability rewards and discounted total rewards as usual. Supporting richer temporal properties is a bit more involved.

We split the algorithmic ideas into

1. operating directly on a finite MDP abstraction,
2. operating over weaker policy classes,
3. operating over stronger policy classes, and
4. operating symbolically in belief space.

4.2.1 Algorithms based on finite MDPsdiscuss!

The main idea for these algorithms is simple: We construct a finite MDP that relates to the POMDP and use standard techniques. In particular, we typically construct a finite MDP that abstracts the infinite belief MDP.

Finite belief MDP unfoldingdiscuss! From a verification perspective, operating immediately on the belief MDP is very convenient. As this MDP is infinite in general, we may explore a finite fragment of this MDP. The *finite belief MDP unfolding* explores $\mathcal{M}^B$ from the initial belief b_{init}, expanding at most K distinct belief states. Any successor belief that falls outside the budget is called a *frontier belief*. To make the finite fragment a proper MDP, we can add transitions from the frontier belief states to either a target or sink state. The way we handle these frontier beliefs is generally controlled by a cutoff function $c\colon S \to [0,1]$.

Definition 4.9 (Finite belief MDP)**.** *Fix a POMDP $\mathcal{M}$, a budget $K \in \mathbb{N}$, and a cutoff function $c\colon S \to [0,1]$. Let $\mathcal{B}^K \subseteq \mathcal{B}$ be the set of at most K distinct beliefs discovered by BFS from b_{init}. The finite belief MDP $\mathcal{M}^{B,K,c}$ agrees with $\mathcal{M}^B$ on all beliefs in $\mathcal{B}^K$, and associates with each frontier belief $b_f \notin \mathcal{B}^K$ a single absorbing "cut" action:*

$$\delta^{B,K,c}(b_f, \text{cut}, T^B) = c \cdot b_f, \qquad \delta^{B,K,c}(b_f, \text{cut}, \bot) = 1 - c \cdot b_f, \tag{133}$$

where $c \cdot b_f = \sum_{s \in S} c(s)\, b_f(s)$ and $\bot$ is a fresh absorbing sink state.

Theorem 4.10 (Over- and under-approximation)**.** *Let V^* be the optimal reachability probability in $\mathcal{M}^B$, and $V^{K,c}$ the optimal reachability probability in $\mathcal{M}^{B,K,c}$.*

- *If $c(s) \geq V^*(s)$ for all $s \in S$: $V^{K,c}(b_{\text{init}}) \geq V^*(b_{\text{init}})$ (over-approximation).*
- *If for some policy π, $c(s) \leq V^\pi(s)$ for all $s \in S$: $V^{K,c}(b_{\text{init}}) \leq V^*(b_{\text{init}})$ (under-approximation).*

In particular, $c \equiv 1$ is always optimistic and $c \equiv 0$ is always pessimistic. As $K \to \infty$, both bounds converge to $V^*(b_{\text{init}})$.

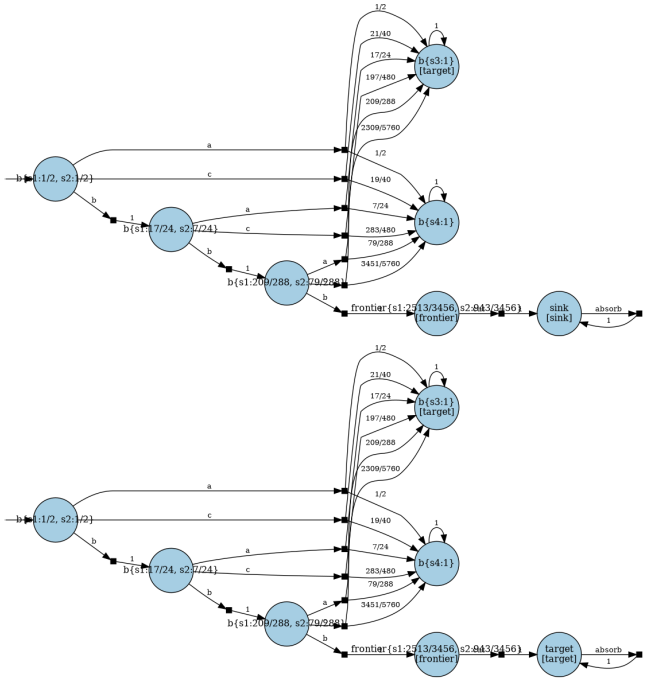
We note the asymetry: Upper bounds are correct using the convexity, whereas the lower bounds must consistently under-approximate a single policy (to then also be affine and thus convex).

Note that computing such cut-offs is simple:

- For lower bounds, picking π and evaluating any (e.g., memoryless) observation-based policy yields a pessimistic bound.
- For upper bounds, assuming the POMDP is fully observable yields an optimistic bound.

However, the computational challenge is to find good cut-off policies with limited overhead. We discuss some strategies to do so Lower bounds from memoryless policiesdiscuss! and Upper bounds from revealing informationdiscuss!, respectively.

Example 4.9. *We construct the finite belief MDP for the 4-state POMDP with initial belief* $b_{\text{init}} = \{s_1 \mapsto \frac{1}{2},\ s_2 \mapsto \frac{1}{2}\}$.



With a restricted budget, frontier beliefs appear. The pessimistic ($c = 0$) *and optimistic* ($c = 1$) *unfoldings give lower and upper bounds on* V^**:*

```
Lower bound: 0.7256944444444444
Upper bound: 1.0
```

The finite exploration above is general, but practically only works well if the (reachable fragment of the) belief MDP is small or in presence of good cut-off values.

Lovejoy grid discretisationdiscuss! In a verification context, proving that no policy achieves a target with a probability above a threshold is arguably the most important task. We note that finite belief MDP unrollings tend to converge slowly to useful upper bounds. On typical problem is that every step in a belief MDP may only change the belief a tiny bit, so belief MDPs quickly grow out of reasonable bounds. The following complementary approach constructs a finite MDP that yields overapproximations without relying on cut-off values. The key idea is to ensure finite exploration by fixing a finite grid (a point set) $\hat{\mathcal{B}} \subset \mathcal{B}$ upfront. Any belief $b' \notin \hat{\mathcal{B}}$ that arises as a successor is not added directly to the state space; instead the belief is represented as a convex combination of the nearest grid points.

The value of such a belief that is not in the point set can be approximated from above by using the convexity of the value function.

Definition 4.10 (Convex grid interpolation)**.** *Let $\hat{\mathcal{B}} = \{\hat{b}_1, \ldots, \hat{b}_m\}$ be a finite set of beliefs whose convex hull covers $\mathcal{B}$. For any belief $b' \in \mathcal{B}$, write $b' = \sum_{i=1}^{m} \lambda_i \hat{b}_i$ with $\lambda_i \geq 0$, $\sum_i \lambda_i = 1$. The grid-interpolated value is*

$$\hat{V}(b') = \sum_{i=1}^{m} \lambda_i \hat{V}(\hat{b}_i). \tag{134}$$

Because V^* is convex, the interpolated value is an over-approximation:

$$V^*(b') = V^*\left(\sum_i \lambda_i \hat{b}_i\right) \leq \sum_i \lambda_i V^*(\hat{b}_i) \leq \hat{V}(b'). \tag{135}$$

This value-level observation motivates a concrete finite MDP construction: rather than storing off-grid beliefs as states, redirect every transition that would land on $b' \notin \hat{\mathcal{B}}$ to a distribution over grid beliefs, using the same convex weights λ_i.

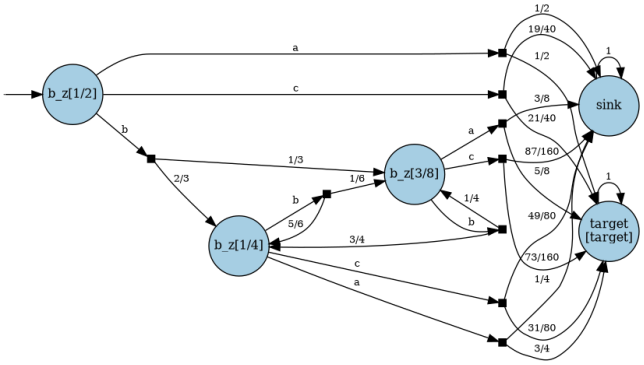
Definition 4.11 (Lovejoy grid MDP)**.** *Let $\hat{\mathcal{B}}$ be a finite grid whose convex hull covers $\mathcal{B}$. For each $b' \in \mathcal{B}$, fix convex weights $\lambda(b', \hat{b}') \geq 0$ with $\sum_{\hat{b}' \in \hat{\mathcal{B}}} \lambda(b', \hat{b}') = 1$ such that $b' = \sum_{\hat{b}' \in \hat{\mathcal{B}}} \lambda(b', \hat{b}')\, \hat{b}'$, where $\lambda(\hat{b}', \hat{b}') = 1$ for on-grid beliefs. The Lovejoy grid MDP $\hat{M}$ has state space $\hat{\mathcal{B}}$ and transition function*

$$\hat{\delta}(\hat{b}, a, \hat{b}') = \sum_{b' \in \mathcal{B}} \delta^B(\hat{b}, a, b') \cdot \lambda(b', \hat{b}'). \tag{136}$$

Theorem 4.11. *The maximal reachability probability in $\hat{M}$ is an upper bound on $V^*(b_{\text{init}})$.*

We remark that building the convex weights is not always trivial. However, when there are only two states per observation, computing these weights becomes reasonably straightforward using linear interpolation.

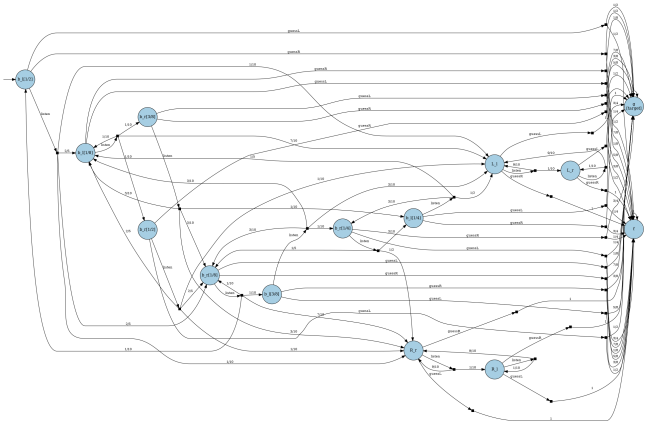
Example 4.10. *We apply the Lovejoy construction to the 4-state POMDP (Example 4.6) with initial belief $b_{\text{init}} = \{s_1 \mapsto \frac{1}{2}, s_2 \mapsto \frac{1}{2}\}$ and grid resolution $k = 8$.*



```
Lovejoy upper bound: 0.75
```

Increasing k refines the grid and tightens the upper bound.

Example 4.11. *We apply the Lovejoy construction to the supremum-not-attained POMDP (Example 4.5). Note that the upper bound here is trivially one (as it is also the supremum). However, the example nicely shows the approximation. We take the initial belief $b_{\text{init}} = \{L_l \mapsto \frac{1}{2}, R_l \mapsto \frac{1}{2}\}$ and grid resolution $k = 8$.*



```
Lovejoy upper bound: 1.0
```

4.2.2 Lower bounds from memoryless policiesdiscuss!

The optimal policy for a POMDP may in general require unbounded memory. A natural restriction is to *memoryless* policies, which choose an action based only on the current observation. We have seen that for some POMDPs, memoryless policies are actually sufficient.

Definition 4.12 (Memoryless observation-based policy)**.** *An observation-based memoryless policy π is a function $\pi\colon Z \to \mathsf{Distr}(A)$.*

Since every memoryless policy is a special case of an observation-based policy:

$$\sup_{\sigma \text{ memoryless}} \mathsf{Pr}^{\sigma}_{M}(\Diamond T) \;\leq\; V^{*}(b_{\text{init}}). \tag{137}$$

Optimising over memoryless policies therefore yields a lower bound on the optimal value. Memoryless policies also often yield helpful bounds for finite belief MDP constructions.

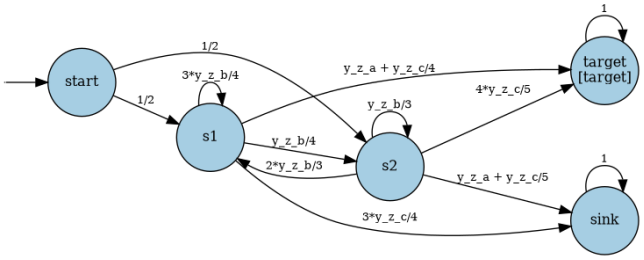
Reduction to a parametric Markov chaindiscuss! A memoryless policy is fully determined by action probabilities $y_{z,a} \in [0,1]$ for each observation $z \in Z$ and action $a \in A_{\mathsf{en}}(z)$, subject to $\sum_a y_{z,a} = 1$. Because the policy is observation-based, all states $s \in S_z$ sharing observation z must use the same distribution, so they share the same parameters. This leads naturally to a

parametric Markov chain (pMC), extending the corresponding pMC for MDPs to the POMDP setting by grouping parameters by observation class.

Definition 4.13 (Corresponding pMC for POMDP)**.** *Extending the corresponding pMC for MDPs, the corresponding pMC D_M for POMDP M has the same state space and introduces parameters $\mathbf{y} = \{y_{z,a} \mid z \in Z,\, a \in A_{\mathsf{en}}(z)\}$, shared across all states with the same observation. The transition function is*

$$\delta_D(s,s') = \begin{cases} \delta(s,a,s') & \text{if } A_{\mathsf{en}}(\mathsf{obs}(s)) = \{a\}, \\ \displaystyle\sum_{a \in A_{\mathsf{en}}(\mathsf{obs}(s))} y_{\mathsf{obs}(s),\,a} \cdot \delta(s,a,s') & \text{otherwise.} \end{cases} \tag{138}$$

Example 4.12. *Applying the transformation to the 4-state POMDP: states s_1 and s_2 share observation z, so they get the same policy parameters.*



Every memoryless policy π corresponds to a well-defined valuation val of $\mathbf{y}$, and vice versa: the induced Markov chain $M[\pi]$ equals the instantiated pMC $D_M[\mathsf{val}]$. The reachability probability $f(\mathsf{val}) = \Pr^{\sigma}_{M}(\Diamond T)$ is therefore a rational function of $\mathbf{y}$, as studied in the parametric chapter.

Theorem 4.12.

$$\sup_{\sigma \text{ memoryless}} \Pr^{\sigma}_{M}(\Diamond T) \;=\; \max_{\mathsf{val} \text{ well-defined for } D_M} f(\mathsf{val}). \tag{139}$$

This reduces finding the best memoryless policy to **parameter synthesis** for pMCs: maximising the rational function f over the policy simplex. The number of parameters is $O(|Z| \cdot \max_z |A_{\mathsf{en}}(z)|)$.

Adding memorydiscuss! Similar to goal unfoldings, specific memorystructures can be folded into the POMDP. Asking for a memoryless policy on this larger POMDP is then equivalent to asking for a finite state controller in a smaller POMDP. While the POMDP grows only polynomially in the number of memory states added, a polynomial growth in the number of parameters makes this quickly infeasible.

Hardness of the problemdiscuss! The existence of a memoryless policy satisfying a reachability threshold is indeed ETR-complete, via the polynomial-time equivalence between this problem and feasibility synthesis for pMCs. Note that these notes do not show the direction from parametric models to POMDPs.

4.2.3 Upper bounds from revealing informationdiscuss!

A natural upper bound on the belief value function $V^*(b)$ is obtained by giving the policies more information, i.e., by optimising over a larger set of policies. Optimising over larger sets of policies can be simpler: The simplest larger class of policies to consider are MDP policies that fully observe the current state. The optimal value in for those policies (i.e., the optimal value in the underlying MDP) is a natural upper bound on the optimal value under observation-based policies. We consider this upper bound and a tighter variation thereof.

MDP upper bounddiscuss! Lifting the fully observable MDP value function V_{MDP} to beliefs by expectation gives

$$V_{\mathrm{MDP}}(b) = \sum_{s \in S} b(s)\, V_{\mathrm{MDP}}(s). \tag{140}$$

This is optimistic: different states in the belief support can implicitly choose different optimal actions, as if the agent knew the hidden state exactly.

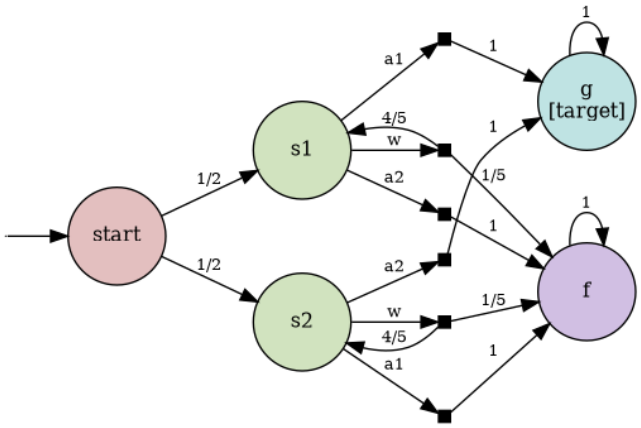
QMDPdiscuss! The *QMDP approximation* requires a single action to be chosen under the current belief, but then assumes the state becomes fully observable for all future decisions:

$$V_{\mathrm{QMDP}}(b) = \max_{a \in A} \sum_{s \in S} b(s) \sum_{s' \in S} \delta(s, a, s')\, V_{\mathrm{MDP}}(s'). \tag{141}$$

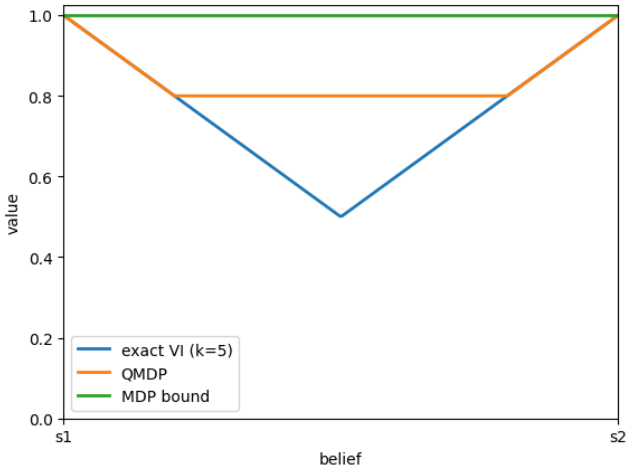
Theorem 4.13. *For every belief $b \in \mathcal{B}$,*

$$V^*(b) \;\le\; V_{\mathrm{QMDP}}(b) \;\le\; V_{\mathrm{MDP}}(b). \tag{142}$$

QMDP is therefore always a tighter upper bound than the plain MDP value.



Example 4.13.



Fast Informed Bounds and Tighter Informed Boundsdiscuss! The idea of V_{QMDP} can be taken further. QMDP assumes full observability after a single action; one can instead delay that assumption for $k \geq 1$ steps. The agent optimises over the next k actions under partial observability — taking into account the observations received — and then falls back to the fully-observable MDP value. Increasing k yields tighter upper bounds at the cost of an optimisation that is exponential in k. Fast informed bounds (FIB) Hauskrecht [2000] and tighter informed bounds (TIB) Krale et al. [2025] are two concrete instantiations of this idea that differ in how the k-step optimisation is carried out.

4.2.4 Value iteration in belief spacediscuss!

The Bellman operator lifts naturally to beliefs:

Definition 4.14 (Belief-space Bellman operator)**.** *The belief-space Bellman operator* Φ *acts on* $V : \mathcal{B} \to [0,1]$ *by*

$$(\Phi V)(b) = \begin{cases} 1 & \text{if } b \in T^B, \\ \max\limits_{a \in A} \sum\limits_{z' \in Z} P(b, a, z') \cdot V(b \mid a, z') & \text{otherwise.} \end{cases} \tag{143}$$

Initialised with the target indicator $V_0(b) = [\text{supp}(b) \subseteq T]$, one application yields the one-step reachability probability, and iterating converges to V^*.

> **Attention**
> Content here is missing.

5 Appendix

5.1 Notationdiscuss!

Let b be a Boolean expression.

$$[b] = \begin{cases} 1 & \text{if } b \\ 0 & \text{otherwise.} \end{cases} \tag{144}$$

We often use the indicator for expressions such as $[x = 0]$.

5.2 Polynomials and rational functionsdiscuss!

Let $X = \{x_0, \ldots, x_n\}$ denote an (ordered) set of variables. A *polynomial* over X is an expression of the shape

$$\sum_i^m c_i x_0^{e_{i,0}} \cdot x_1^{e_{i,1}} \cdot \cdots \cdot x_n^{e_{i,n}} \tag{145}$$

with exponents $e_{i,j} \in \mathbb{N}$ and coefficients $c_i \in \mathbb{Q}$. We denote the set of polynomials as $\mathbb{Q}[X]$.

A *rational function* is a fraction of two polynomials. The set of rational functions is written \ratfunc{X}$.

5.3 Geometrydiscuss!

Let $\vec{p} = (p_1, \ldots, p_m) \in \mathbb{R}^m$ be a point in an m-dimensional Euclidean space. For $c \in \mathbb{R}$, let $c \cdot \vec{p} = (c \cdot p_1, \ldots, c \cdot p_m)$ be a scalar multiplication.

- $R \subseteq \mathbb{R}^m$ is *convex* iff $\vec{p}, \vec{q} \in R$ implies $c \cdot \vec{p} + (1 - c) \cdot \vec{q} \in R$ for all $c \in [0, 1]$.
- $R \subseteq \mathbb{R}^m$ is *downward-closed* iff $\vec{p} \in R$ and $\vec{p} \geq \vec{q} \in \mathbb{R}^m$ implies $\vec{q} \in R$.

We say $\vec{p} \in \mathbb{R}^m$ *dominates* $\vec{q} \in \mathbb{R}^m$, written $\vec{p} \succ \vec{q}$, iff $\vec{p} \geq \vec{q}$ and $\vec{p} \neq \vec{q}$.

5.4 Distributionsdiscuss!

A *discrete distribution* over a set X is a function $\mu \colon X \to [0, 1]$ such that $\sum_{x \in X} \mu(x) = 1$. The *support* of μ is $\mathsf{supp}\,(\mu) = \{x \mid \mu(x) > 0\}$.

A distribution μ is *Dirac*, if $|\mathsf{supp}\,(\mu)\,| = 1$.

5.5 Deterministic finite automatadiscuss!

Definition 5.1. *A deterministic finite automaton is a tuple*

$$\langle Q, \Sigma, q_0, \delta, \mathsf{Acc} \rangle \tag{146}$$

with

- *a finite set of states* Q,
- *a finite alphabet* Σ,
- *an initial state* $q_0 \in Q$,
- *a transition relation* $\delta \colon Q \times \Sigma \times Q$,
- *A set of accepting states* $\mathsf{Acc} \subseteq Q$.

For a given alphabet, a word w is a sequence of alphabet symbols, i.e., $w \in \Sigma^*$. We use ϵ to denote the sequence of length zero.

We lift the transition relation δ inductively to words: $\delta \colon Q \times \Sigma^* \to Q$. Intuitively, we simply follow the transitions by the DFA to arrive at a state. Formally, we inductively define this transition as follows:

- $\delta(q, \epsilon) = q$, and
- $\delta(q, w \cdot a) = \delta(\delta(q, w), a)$.

Definition 5.2 (DFA language)**.** *The language accepted by the DFA is the set of words that end in a final state*

$$\mathcal{L}(\mathcal{A}) = \{w \in \Sigma^* \mid \delta(q_{init}, w) \in F\} \tag{147}$$

.

5.6 Fixpoint theorydiscuss!

5.6.1 Latticesdiscuss!

Let $(L, \leq)$ be a partially ordered set, and let $X \subseteq L$.

- An element $u \in L$ is an *upper bound* of X if $x \leq u$ for all $x \in X$.
- An element $l \in L$ is a *lower bound* of X if $l \leq x$ for all $x \in X$.
- The *least upper bound* (or *join*) of X, denoted $\bigvee X$, is an upper bound u of X such that for any other upper bound u' of X, $u \leq u'$.
- The *greatest lower bound* (or *meet*) of X, denoted $\bigwedge X$, is a lower bound l of X such that for any other lower bound l' of X, $l' \leq l$.

Definition 5.3 ((complete) latttices)**.** *A lattice* $(L, \leq)$ *is a partially ordered set in where each pair* $x, y \in L$ *has a join and a meet in* L.
A lattice is complete, if every subset $X \subseteq L$ *has a join* $\bigvee X$ *and a meet* $\bigwedge X$ *in* L.

5.6.2 Monotone and Continuous Operatorsdiscuss!

Let $(L, \leq)$ be a partially ordered set. Operators are functions from L to L.

Definition 5.4 (monotone operator)**.** *An operator $\Psi\colon L \to L$ is monotone if for all $x, y \in L$:*

$$x \leq y \implies \Psi(x) \leq \Psi(y) \tag{148}$$

Definition 5.5 (-continuous operator)**.** ω
Let $(L, \leq)$ be a complete lattice and $\Psi\colon L \to L$ a monotone operator.
The operator F is ω-continuous if it preserves suprema of countable ascending chains. Formally: Let $(x_i)_{i\geq 0}$ be a countable ascending chain in L, i.e.,

$$x_0 \leq x_1 \leq x_2 \leq \dots \tag{149}$$

then Ψ is ω-continuous if:

$$\Psi\Big(\bigvee_{i\geq 0} x_i\Big) = \bigvee_{i\geq 0} \Psi(x_i) \tag{150}$$

The intuition here is that applying F to the "limit" of an ascending chain is the same as taking the limit of applying F to each element. This property is crucial for infinite lattices, which we encounter in analysing MDPs.

5.6.3 Fixpointsdiscuss!

Definition 5.6 (Fixpoint)**.** *An element $x \in L$ is a fixpoint of $\Psi\colon L \to L$ if* $\Psi(x) = x$

The set of all fixpoints of Ψ is denoted $\mathrm{Fix}(\Psi)$.

Theorem 5.1 (Knaster-Tarski Theorem)**.** *Let $(L, \leq)$ be a complete lattice and $\Psi\colon L \to L$ be monotone. Then:*

1. *Ψ has a least fixpoint $\mathsf{lfp}.\Psi$ and a greatest fixpoint $\mathsf{gfp}.\Psi$.*
   They are given by

$$\mathsf{lfp}.\Psi = \bigwedge\{x \in L \mid \Psi(x) \leq x\}, \quad \mathsf{gfp}.\Psi = \bigvee\{x \in L \mid x \leq \Psi(x)\} \tag{151}$$

2. *If $(x_i)_{i\geq 0}$ is defined by*

$$x_0 = \bot, \quad x_{i+1} = \Psi(x_i) \tag{152}$$

   then $\mathsf{lfp}.\Psi = \bigvee_{i\geq 0} x_i$.

For *finite lattices*, the least fixpoint $\mathsf{lfp}.\Psi$ can be computed by iterating Ψ starting from the least element $\bot$ until a fixpoint is reached. Similarly, the greatest fixpoint $\mathsf{gfp}.\Psi$ can be obtained by iterating from the greatest element $\top$.

References

C. Baier and J.-P. Katoen. *Principles of model checking*. MIT Press, 2008.

K. Chatterjee, T. Quatmann, M. Schäffeler, M. Weininger, T. Winkler, and D. Zilken. Fixed Point Certificates for Reachability and Expected Rewards in MDPs. In *TACAS (2)*, Lecture Notes in Computer Science, pages 130–151. Springer, 2025.

V. Forejt, M. Z. Kwiatkowska, G. Norman, D. Parker, and H. Qu. Quantitative Multi-objective Verification for Probabilistic Systems. In *TACAS*, Lecture Notes in Computer Science, pages 112–127. Springer, 2011.

S. Haddad and B. Monmege. Interval iteration algorithm for MDPs and IMDPs. *Theor. Comput. Sci.*, 735:111–131, 2018.

M. Hauskrecht. Value-Function Approximations for Partially Observable Markov Decision Processes. *J. Artif. Intell. Res.*, 13:33–94, 2000. doi: 10.1613/JAIR.678.

M. Krale, W. Koops, S. Junges, T. D. Simão, and N. Jansen. Tighter Value-Function Approximations for POMDPs. In *AAMAS*, pages 1200–1208, 2025. doi: 10.5555/3709347.3743641.

T. Quatmann. Verification of multi-objective Markov models, 2023.

J. Von Neumann. Various techniques used in connection with random digits. *Appl. Math Ser*, 12(36-38):5, 1951.